Parallel Search for Alien Worlds: GPU-Accelerated BLS and TLS for Exoplanet Transit Detection

Mauri de Souza Meneguzzo

ABSTRACT

A transit survey repeats its search on every star: with planets injected, to measure what it can find, and on shuffled or inverted copies, to measure its false alarms. The work is one operation repeated over stars, trial periods and copies, which a GPU executes efficiently. We present transitgpu, a Python package that runs three transit searches on a GPU through PyTorch and Triton (CUDA and ROCm): a box search (BLS) whose integer sums do not depend on the order of the additions, a port of Transit Least Squares (TLS) 2.0.1 that keeps its interface and results, and a hybrid that refines the highest peaks of the box spectrum with TLS. The TLS port has an exact mode, which reproduces the χ^2 of TLS at every trial period to float64 rounding on 25 test light curves, and a faster binned mode that approximates it. On an AMD Radeon RX 7900 XTX at 2×10^5 points, exact mode takes 9.91 s, binned mode 1.48 s and the box search 0.477 s; TLS on 16 CPU threads takes 27.7 s with its exact backend and 10.8 s with its default one. In injection tests on three classes of synthetic noise and on one sector of each of 12 quiet TESS stars, at a matched false-alarm rate, binned mode finds as many planets as exact mode within 1.4 points. The box search and the hybrid match exact mode in white noise and on the quiet stars and fall behind in correlated noise and on a rotating star. The code is public under the MIT licence.

Keywords: Exoplanet detection methods (489) — Transit photometry (1709) — Astronomy data analysis (1858)

1. INTRODUCTION

A transit search folds a light curve at each of many trial periods and scores how well a dip explains the folded points, so it costs about one pass over the points per trial period. A stack of several years of TESS (Ricker et al. 2015) data at 2- to 30-minute cadence has about 2×10^5 points, and the period grid of Ofir (2014) for such a baseline has about 10^5 trial periods. Box Least Squares (BLS; Kovács et al. 2002) fits a box-shaped dip. Transit Least Squares (TLS; Hippke & Heller 2019) fits a limb-darkened template, which makes it more sensitive at low signal-to-noise ratio but slower. On a light curve of 2×10^5 points and 91 607 trial periods, TLS 2.0.1 takes 10.8 s on 16 threads and 85.6 s on one with its default backend, and 27.7 s and 277 s with its exact backend (Sect. 4.6). Repeated searches multiply this cost. The Kepler catalog of Thompson et al. (2018) measured its false alarms by running the search on inverted and scrambled copies of every light curve, Kunimoto et al. (2023a) searched 92 899 TESS M dwarfs and their inverted light curves, and injection tests plant artificial transits and run the search once per planted planet to measure what it can find. A per-star false-alarm threshold from shuffled copies of a light curve, the method of Sect. 3.4, runs the search once per copy; with the 48 copies that are the default here, that is 49 searches of every star.

GPU searches already exist. The TESS Quick-Look Pipeline (QLP) has searched its multi-sector light curves with the GPU BLS of cuvarbase (Hoffman 2022) since Sector 59. On the multi-sector light curves of one CCD of each of two cameras in Sector 58, the GPU search took 1.9 and 4.2 s per star, against 77 and 172 s for the CPU BLS of VARTOOLS (Hartman & Bakos 2016), which searched an under-sampled grid; it recovered 392 of 468 TESS Objects of Interest (TOIs), against 380 (Kunimoto et al. 2023b). GTLS (Hu et al. 2026) ports TLS to NVIDIA GPUs with CuPy and uses the GPU BLS of cuvarbase as a timing baseline. CETRA (Smith et al. 2025) detects single transits in linear time on NVIDIA GPUs and then combines them into periodic signals with a physical transit model. On synthetic white-noise light curves of 173 days with periods of 1 to 86.5 days, its authors report at least 20% more low-S/N transits recovered than with TLS, particularly at long periods, and run times up to a few orders of magnitude shorter for high-cadence light curves. All three are written for NVIDIA GPUs; Sect. 5 compares GTLS with `transitgpu.tls`.

This paper describes transitgpu, which runs transit searches through PyTorch (Paszke et al. 2019) and Triton (Tillet et al. 2019), both of which target CUDA and ROCm. A user gets three searches and a batch call. The *box search* is a phase-binned BLS with integer sums, whose bin sums do not depend on the order of the additions (Sect. 2.3). The

TLS search, `transitgpu.tls`, keeps the interface, grids, template and results objects of TLS, so code written for TLS runs on a GPU after a change of import, and has an *exact mode* and a faster, approximate *binned mode* (Sects. 2.4 and 2.6). The *hybrid search*, `transitgpu.bls_tls_hybrid`, runs the box search over the whole grid and exact mode on narrow windows around its highest peaks (Sect. 2.7). The batch call searches many light curves, or a light curve and its shuffled copies, in one launch. A biweight detrend computes wotan's trend on the GPU (Sect. 2.8). TLS 2.0.1 itself has an exact backend and a default one that replaces long templates with piecewise-linear fits; following their names in the code, we call them TLS fused and TLS fused-pl.

The exact mode is TLS 2.0.1's exact search reproduced on a GPU with its results kept: the permutation of a stable float64 sort of the phases and the rounding allowances of the screening are those of TLS (Sects. 2.5 and 4.1). The screening cascade has one bound that is added here, U_2. The bounds U_1 and U_3 come from TLS 2.0.1, and the data layout follows GTLS. At each trial period every search scores each *trial epoch*, the phase at which a transit window starts, for a set of durations.

On one consumer AMD GPU, the box search searches a light curve of 2×10^5 points and 48 shuffled copies of it in one call in 9.79 s, less than the 27.7 s of one search with TLS fused on 16 CPU threads, although the box statistic is not that of TLS. Binned mode searches the whole grid with the TLS template in 1.48 s. In three classes of synthetic noise and on 12 quiet TESS stars, one sector each, it finds as many injected planets as exact mode within 1.4 points at a matched false-alarm rate. Exact mode takes 9.91 s with the same χ^2 as TLS to float64 rounding. Section 4 gives these measurements with their conditions.

Section 2 describes the algorithms, Sect. 3 the data; Sect. 4 compares the searches with TLS and times them against TLS and astropy's `BoxLeastSquares` (Astropy Collaboration et al. 2013, 2018, 2022), Sect. 5 compares the TLS search with GTLS, Sect. 6 interprets the results together, Sect. 7 states the conclusions and their limits; Appendix A holds the tables and figures of the validation and the timing, and Appendix B the command that reproduces each measured number.

2. ALGORITHMS

2.1. *The box statistic*

Let $y_i = f_i - \bar{f}$ be the flux minus its mean over N points. For a window that holds n points and has a lower mean than the whole light curve, the box statistic with equal errors σ is

$$S = \frac{\left(\sum_{i \in \text{window}} y_i\right)^2}{\sigma^2 \, n \, (1 - n/N)} \tag{1}$$

(Kovács et al. 2002). It is the square of the signal-to-noise ratio of the depth when the level outside the box is free, and so the square of the `snr` objective of astropy's `BoxLeastSquares`. The default `likelihood` objective of astropy is $\frac{1}{2}\,\text{ivar}_\text{in}\,\text{depth}^2$, which differs from $S/2$ by the factor $(1 - n/N)$. The depth of the best box, positive for a dip, is the mean flux outside minus the mean inside,

$$\delta = -\frac{\sum y_i}{n \, (1 - n/N) \, \bar{f}}, \tag{2}$$

and its signal-to-noise ratio is $\delta \bar{f} \sqrt{n(1 - n/N)}/\sigma$, whose square is S. Let $S(P)$ be the best S at trial period P. The signal detection efficiency (SDE) of the box search is the z-score $(\max S - \langle S \rangle)/\text{std}(S)$, with the mean and standard deviation taken over the trial periods. Unlike the SDE of TLS, it is not detrended.

2.2. *TLS and GTLS*

TLS replaces the box with a limb-darkened transit template computed with batman (Kreidberg 2015). For each period and duration it sorts the folded points by phase and slides the template over every trial epoch. Its SDE is also a z-score over the grid, $\text{SDE}_\text{raw} = (1 - \langle SR \rangle)/\text{std}(SR)$ with $SR = \chi^2_\text{min}/\chi^2$, which TLS then detrends with a running median. Besides TLS fused-pl, its default, and TLS fused, TLS 2.0.1 keeps its original backend, which uses Numba (Lam et al. 2015).

GTLS (Hu et al. 2026) keeps this algorithm on an NVIDIA GPU with CuPy; Sect. 5 describes it.

2.3. *The box search*

The box search is written in PyTorch. It runs on a GPU, and on the CPU when torch finds no GPU or when the call names the CPU; the tests use the CPU run as a reference. It uses the period grid of TLS (solar star, oversampling 3, at least 2 transits, uniform in $f^{1/3}$ for frequency f; Ofir 2014). At each period it tries durations from the shortest to the longest that TLS allows with its default stellar limits, on a geometric grid of ratio 1.1, which is the step of TLS, plus one step beyond the longest. For each batch of periods it proceeds as follows.

1. *Fold.* The phase of every point, its time multiplied by the trial frequency $1/P$, is computed in float64, scaled to $B(P)$ bins and truncated. A product is used rather than t/P because float64 division is slow on GPUs; it changes the phase by at most a unit in the last place. $B(P)$ depends only on the period: it is the power of two that puts at least 12 bins across the shortest duration at that period, clipped to 128–4096. The batch size therefore changes the run time but not the bin sums; the scores computed from them can differ in their last digits between batch sizes (Sect. 4.3).

2. *Accumulate.* Each bin sums y and counts its points. The flux is quantized to 10^{-9} and summed as integers, because a floating-point sum on a GPU depends on the order of its atomic additions and integer sums are exact in any order; a flux whose 64-bit sums could overflow is rejected. On ROCm a HIP kernel performs this step: each block folds one period over a range of points into a private table of 64-bit sums and 32-bit counts in the local memory of its compute unit, then merges the non-zero bins into the global tables with one atomic addition each. It serves every call on ROCm that has up to 4096 bins and weights shared by all the light curves of the call: the box search of a single light curve (the box and hybrid rows of Table 6) and the call that searches several light curves (`search_many`, the 64 light curves and the 48 shuffled copies of Sect. 4.6). It gives the same integers as the Triton kernel, which serves CUDA and every other case.

3. *Score.* Cumulative sums over the bins, wrapped around the cycle, give every window of every duration. Eq. (1) is evaluated for windows that are dips and have at least 3 points inside and 3 outside. The scores are computed in 32-bit floats from the exact integer sums; 64-bit scores are an option.

4. *Reduce.* The best window of each period is kept. The SDE, the period uncertainty (the half width at half maximum of the peak, as TLS defines it), the epoch, the duration, the depth and the signal-to-noise ratio follow, under the names that TLS uses.

Nothing is sorted and no template is slid: the work per period is one pass over the points plus a few hundred bins.

With errors dy_i, the points are weighted. Each inverse-variance weight $w_i = 1/dy_i^2$ is scaled by the median weight and stored as the integer $W_i = \mathrm{clip}(\mathrm{round}(64\, w_i/\mathrm{median}\, w), 1, 4096)$, so that the sums stay integers. The level is the weighted mean, the bins hold $\sum W_i y_i$ and $\sum W_i$ instead of $\sum y_i$ and the counts, and a window with weight W_in out of a total W scores

$$S = \frac{\left(\sum_{i\in\mathrm{window}} W_i y_i\right)^2}{\sigma_W^2\, W_\mathrm{in}\,(1 - W_\mathrm{in}/W)}, \tag{3}$$

with $\sigma_W^2 = 64 \sum_i W_i (y_i - \bar{y})^2 / \sum_i W_i$. Because the SDE is a z-score over periods, the scale of σ_W cancels. The minimum of 3 points inside and outside counts points, not weights. Equal weights reduce to the unweighted search, in which one σ serves the whole light curve.

Two options reuse the fold. With `bumps=True`, the windows whose mean lies above that of the whole light curve are scored from the same bin sums, and each result also holds the best bump, equal to the best dip of a search of the negated flux; the dips are scored as they are without the option. Given one flux and a matrix of weights, `search_many` searches the flux once for each row from one fold of the times, and a weight of zero masks its point. Neither option is used in the measurements of this paper.

2.4. *Exact mode*

`transitgpu.tls` leaves the period grid, duration grid, transit model, statistics and final fit to TLS 2.0.1 and replaces only its loop over trial periods with code in PyTorch and Triton. Its class `transitleastsquares` is that of TLS with a different default backend. TLS hands its backend the trial periods in a shuffled order by default (`PERIODS_SEARCH_ORDER` in its `tls_constants.py`); exact mode sorts them before it batches them, because the cost of a batch follows the union of the template widths of its periods, and returns the results in the order it was given,

so the order changes nothing in them. The data layout on the GPU follows the source code of GTLS (gputls 0.4.4), reimplemented in PyTorch; Sect. 5.1 says what release 0.5.1 changes.

For a template s_j of m samples starting at sorted point i, the χ^2 of TLS is the residual of the constant model plus $2k_i\mathcal{B}_i + k_i^2\mathcal{C}_i$, where k_i is the depth scale at that trial epoch,

$$\mathcal{B}_i = \sum_{j=0}^{m-1} w_{i+j}(d_{i+j} - 1)(1 - s_j), \qquad \mathcal{C}_i = \sum_{j=0}^{m-1} w_{i+j}(1 - s_j)^2, \tag{4}$$

d is the folded flux and w the inverse-variance weight. $\mathcal{B}$ and $\mathcal{C}$ are correlations of the template with the folded data, evaluated at the trial epochs that TLS tests: those where the mean flux in the window is deeper than the shallowest depth fitted. With equal errors, $\mathcal{C}$ depends only on the template. Four steps find the minimum over trial epochs and durations for a batch of periods.

1. *Fold.* TLS orders the points of a period by their float64 phase, with ties in index order. On the GPU, a counting sort puts the points into 2^q phase bins of two to four points each and ranks each point within its bin by phase and index; a period with a bin of more than 256 points falls back to a stable comparison sort. Either way the result is the permutation of a stable float64 sort.

2. *Prefix sums.* With the flux in that order and wrapped around by the length of the longest template, one kernel computes the cumulative sums of the flux, of $w(f-1)^2$ and of the centred weighted residual, the cumulative sum of that last sum (the double prefix sum), and the residual of the constant model. A first pass sums chunks of 1 024 points, a short cumulative sum gives the carry of each chunk, and a second pass scans each chunk and adds its carry.

3. *Screen.* The cascade of Sect. 2.5 selects the trial epochs that need an exact correlation.

4. *Hand back.* The χ^2, the template row and the depth at the best trial epoch of each period go to TLS, which computes the periodogram, the SDE, the uncertainties and the final fit as it does for its own backends.

When the errors differ between points, the search computes $\mathcal{B}$ and $\mathcal{C}$ by direct correlation in float64 instead of screening. A third backend, `gpu-pl`, runs the default statistic of TLS, fused-pl, on the GPU with equal errors; Table 6 times it, and its recovery is not evaluated here.

2.5. *The screening cascade*

Write $r = 1 - d$ for the residual and $A_i = -\mathcal{B}_i = \sum_j w_{i+j} r_{i+j}(1 - s_j)$ for its correlation with the template. The gain of trial epoch i over the constant model is then $G_i = 2k_iA_i - k_i^2\mathcal{C}_i$. A trial epoch can lower the minimum only if its gain exceeds the best gain found so far at its period, the incumbent. The incumbent of a period starts as the best gain among the trial epochs where a box of each template width is deepest, taken over every eighth trial epoch. Three upper bounds on A_i, and hence on the gain, need no sum over the template:

1. U_1, the Cauchy–Schwarz bound $A_i \le \sqrt{\mathcal{C}_i(R_i + \epsilon)}$, where R_i is the sum of wr^2 over the window, a difference of prefix sums, and ϵ is an allowance for the rounding of those prefix sums that keeps the screen conservative;

2. U_2, a coarse piecewise-linear proxy for the template with few pieces, whose correlation with the residual is a sum of as many terms of the double prefix sum as it has pieces;

3. U_3, the proxy of TLS, with about 15 pieces.

U_1 and U_3 come from TLS 2.0.1. U_2 and U_3 add to the correlation of each proxy a term for the distance between the template and the proxy, with the same allowances for rounding as TLS. The bounds are applied in the order U_1, U_2, U_3, and a trial epoch is dropped as soon as its bound on the gain falls below the incumbent. The survivors get their exact dot product in float64, so their χ^2, template row and depth are those of TLS fused. Among the survivors the highest gain wins; on equal gains the earlier width and then the earlier trial epoch win, as the strict comparisons of TLS in ascending order give.

2.6. *Binned mode*

Binned mode folds the points of each period into N_b phase bins with the integer accumulation used by the box search, without sorting. It averages the template of each duration over the bins and obtains A, $\mathcal{C}$ and the window sums as circular correlations of the bin sums and counts with the binned template, its square and a box of the duration, using Fourier transforms of length N_b. N_b is the smallest power of two for which N_b times the shortest allowed duration, as a fraction of the period, reaches a chosen number of bins, taken at the longest period that has the same N_b. The periods of a search are split into the bands that share an N_b (with TLS's default stellar limits and 12 bins across the shortest duration: 4 096 bins at 10 to 14 d, 8 192 at 30 and 50 d, 32 768, the cap, at 200 d and longer), and each band is searched with its own bin count and its own binned templates.

The statistic equals that of TLS up to the smoothing of data and template over one bin. It departs from TLS where the folded points cover the phase unevenly (few sectors, gaps, one or two transits), because a TLS window holds a fixed number of points whereas a bin window holds a fixed width in phase. Binned mode needs equal errors. On the 14 characterization cases of TLS (Sect. 3.1), with TLS fused as the reference, it is less accurate than TLS fused-pl. It rejects the 2 cases with unequal errors. On the other 12 it finds a different best period in 2 (a light curve with a gap and one of 150 points), and its SDE differs by at most 0.87. TLS fused-pl finds the best period of TLS fused in all 14 cases, with SDE differences of at most 0.065. Table 11 (Appendix A) gives the cases.

2.7. *The hybrid search*

`transitgpu.bls_tls_hybrid` runs the box search over the whole period grid and then calls the `power()` method of TLS on a window within 1% in period of each of the n_{peak} highest peaks in the box spectrum, with exact mode and $n_{\mathrm{peak}} = 5$ by default. A peak is a local maximum of the spectrum, and peaks closer together than the window count as one. The function returns the box spectrum and its SDE and, for each peak in rank order, the period, the SDE and the TLS results object, which refines the period, epoch, duration and depth with the template. A peak that TLS cannot refine keeps the box values and records the reason. Table 1 compares the design of the searches with TLS and GTLS.

Table 1. Design of the searches. TLS and GTLS as described by Hippke & Heller (2019) and Hu et al. (2026).

	TLS	GTLS	transitgpu box search	`transitgpu.tls`
transit model	limb-darkened template	as TLS	box	as TLS
per period	sort by phase, slide template	sort by phase, template on GPU	bin the phase, sum windows	exact mode: sort by phase, screen trial epochs; binned mode: bin the phase
period grid	Ofir (2014)	Ofir (2014)	as TLS	as TLS
duration limits	TLS defaults	the limits of TLS in its source (the paper describes wider ones)	TLS defaults	as TLS
trial epochs	all	coarse, then all for ∼200 candidates	all bin windows	all that TLS tests
SDE	z-score, detrended with a running median	as TLS	z-score, not detrended	as TLS
hardware	CPU	NVIDIA GPU (CuPy)	GPU (PyTorch)	GPU (PyTorch and Triton)

2.8. *Detrending*

`transitgpu.detrend.biweight` computes the trend of the biweight filter of wotan (Hippke et al. 2019) on the GPU, with wotan's windows, mask and stopping rule. The window of a point at time t holds the points with

$t - L/2 \le t_i < t + L/2$ for a window length L. The light curve is cut into segments where consecutive times differ by more than $L/2$, and no window crosses a segment. The points of a mask, such as known transits, are left out of every window. The trend of a window is the biweight location: the median, then steps of a weighted mean with weights $(1 - u^2)^2$ for $|u| < 1$ and $u = (x - c)/(5\,\mathrm{MAD})$, until a step is at most 10^{-6}. On a CUDA or ROCm device a Triton kernel runs one program for each point, reads the point's window from the flux array, and sorts it in registers for the median and the median absolute deviation. Elsewhere torch operations on a matrix of windows compute the same trend, on a wider window of the GPU and on the CPU. On the CPU the torch version is faster than wotan at 10 minutes and 200 seconds of cadence and slower at 2 minutes (Sect. 4.7). `biweight_many` computes the trends of several light curves, such as the sectors of one star, in one sequence of launches. The tests compare both versions with wotan 1.10 to a relative 10^{-9}, with equal positions of the NaN values, on light curves with gaps, NaN values, masks, segments of one to three points and gaps of half the window.

2.9. *Implementation*

Figure 1 gives the architecture: what runs on the host, what runs on the device, and the code of each stage. The package exposes four entry points: `box_search` for one light curve, `search_many` for several light curves with shared weights, or a light curve and its shuffled copies, `transitgpu.tls.transitleastsquares`, the class of TLS with the GPU backends (`gpu`, exact mode, the default, and `gpu-binned`), and `bls_tls_hybrid`. A search written for TLS runs on the GPU after one changed import:

```
from transitgpu.tls import transitleastsquares
results = transitleastsquares(t, y, dy).power(period_min=0.5, period_max=5.0)
```

The searches run on the first GPU that torch finds, on the CPU without one, or on the device that a call names; `transitgpu.engine` holds the same searches with the device as an argument. On the CPU the fold, the scoring and the update of the best boxes are compiled with `torch.compile`, as on a GPU, and the backend `gpu-pl`, which has only Triton kernels, refuses to run on the CPU. The accumulation of the box search runs in a HIP kernel on ROCm, compiled with `hipcc` at the first call of an environment and cached; without `hipcc`, or on CUDA, the Triton kernel serves, and without Triton the fold is compiled with `torch.compile`. Exact mode batches up to 256 trial periods by the free device memory and halves a batch whose mask of trial epochs would pass 2^{30} entries until it fits; a batch that still passes, which only a period with 2^{30} trial epochs of its own can, is refused with an error that names both numbers; a flux whose 64-bit integer sums could overflow is rejected. The tests check each kernel against a NumPy or torch reference, the screened search against a brute-force search over all trial epochs, and `transitgpu.tls` against TLS fused on the 14 characterization cases field by field; the continuous-integration suite runs on the CPU, where the tests that need a device are skipped.

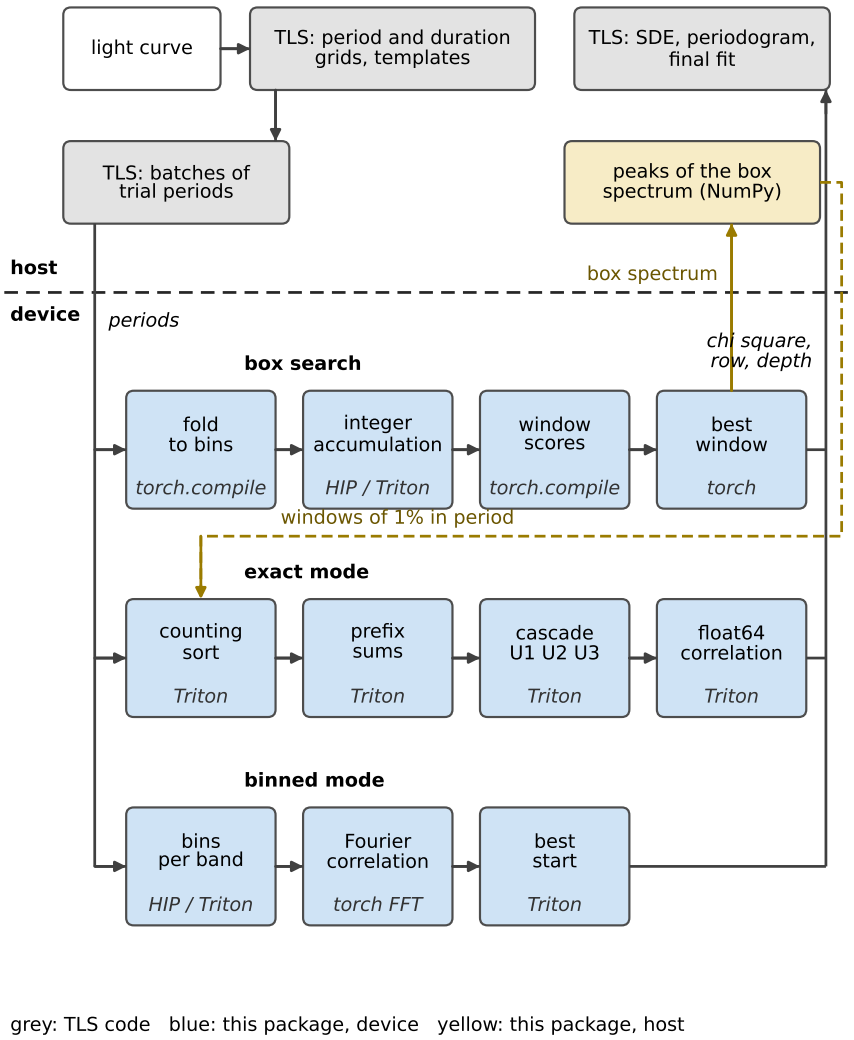


Figure 1. The architecture of the package. Above the line, the host: the light curve, the period and duration grids and the templates of TLS, the loop over batches of trial periods, and the periodogram, the SDE and the final fit, grey where the code is TLS 2.0.1's. Below the line, the device: the three paths of a batch, the box search (the fold to bins, the integer accumulation, the window scores from cumulative sums), exact mode (the counting sort, the prefix sums, the cascade U_1, U_2, U_3, the float64 correlations of the survivors) and binned mode (the bins of each band, the Fourier correlation, the best start), with the code that runs each stage at its arrow (the HIP kernel on ROCm or the Triton kernel, Triton, `torch.compile`, the Fourier transform of torch). The hybrid search is the loop from the peaks of the box spectrum, found on the host, back into exact mode on narrow windows. The names are those of Sect. 2.3 to 2.7.

3. DATA AND METHODS

3.1. *Characterization cases*

TLS 2.0.1 ships 14 characterization cases: short runs on synthetic and K2 (Howell et al. 2014) light curves with different grids, templates, errors and gaps. The two K2 light curves are EPIC 201367065 and EPIC 206154641. We run each case with TLS fused and with exact mode and compare the results field by field.

3.2. *Benchmark input*

The benchmark times every search on one input with one period grid. The light curve has $N = 2 \times 10^4$, 10^5 or 2×10^5 points evenly spaced over 1 000 days, flat flux with Gaussian noise of 1 000 ppm, and one injected box with a period of 3.18 d, a depth of 3 000 ppm and a duration of 2.5 h, drawn from a fixed seed. The period grid is that of TLS from 0.5 to 14 days, with 91 607 trial periods. The searches do different amounts of work per period: the box search scores about 24 durations on average, astropy's `BoxLeastSquares` one list of 37 durations, and TLS a template at 38

durations limited by the stellar limits at each period. For the search of many light curves, 64 light curves of 2×10^4 points are each given their own noise. Longer grids use 72 000 points over 1 500 days and 144 000 points over 3 000 days, with periods up to half the baseline; these are the sizes of the GTLS benchmark (Hu et al. 2026).

3.3. *Real TESS light curves*

The recovery of injected planets is measured on synthetic light curves (Sect. 4.4) and on one sector of the 2-minute PDCSAP light curve of each of 12 quiet TESS dwarfs, three each of F, G, K and M type by the effective temperature of the TESS Input Catalog (F 6 000 to 7 300 K, G 5 300 to 6 000 K, K 3 900 to 5 300 K, M below 3 900 K) with $\log g$ of at least 4.0. A star was taken if it is not in the TOI table (snapshot of 2026-10-21), has CROWDSAP of at least 0.99, a TESS magnitude from 8.5 to 12, at least 17 500 cadences with quality flag 0, and a 5-to-95 percent range of its flux binned to 0.5 days of at most twice what its white noise gives; the first three of each type that pass, in a seeded order, were taken. The 12 are from sectors 1, 2, 20, 21, 39, 41, 89 and 97, with TESS magnitudes from 8.6 to 12.0 and a white-noise scatter from 531 to 3 167 ppm; `validation/stars.json` lists each with its TIC number, sector, effective temperature, file and SHA-256, which `validation/fetch.py` downloads and verifies. The planets of Sect. 4.4 are injected into each star, 3 at each of the 30 cells of period and signal to noise (1 080 in all), and the light curve is then flattened with the same biweight filter as the synthetic classes. The injected transits are those of a planet around a star of the solar density on every star, as in the synthetic classes, and the duration grid is TLS's default for a solar star, so on the K and M dwarfs the durations are longer than those of a physical transit of theirs; the test measures the searches on real noise, not on physical durations. The errors given to every search on a real star are the median of the reported errors of its sector, so that binned mode, which needs equal errors, runs on the same input as the others; the real-star test therefore does not exercise per-point weights. The null set of each star is the star itself and 100 copies of it shuffled in blocks of one day after flattening.

3.4. *Per-star thresholds from block shuffles*

The SDE of the box search is a z-score over the period grid of one light curve and has no threshold of its own. A threshold per light curve can be set by the largest SDE among n copies of it, each shuffled in blocks within each sector; the block length and n are parameters, and the defaults, used throughout this paper, are blocks of one day and $n = 48$. The count sets the smallest false-alarm rate the threshold can resolve, $1/(n+1)$, and its cost, n searches; 48 is the default because it resolves a rate of about 2%, the rate at which the calibration of Sect. 4.5 was tested, and no other value of n is compared here. If the shuffled copies and a noise-only light curve are exchangeable, a noise search exceeds the threshold in 1 search in $n+1$, and the k-th largest SDE corresponds to a rate of $k/(n+1)$: at the default, 1 in 49 (2.04%). The threshold is itself a random variable: the false-alarm probability of the threshold of one light curve follows a Beta$(1, n)$ distribution, at the default Beta$(1, 48)$, with a median of 1.4%, a 90th percentile of 4.7% and a 99th percentile of 9.1%, and the tail beyond 1 in 49 is not resolved. The box search folds a light curve and its shuffles together in one call; Sect. 4.6 gives the cost, and Sect. 4.5 tests the exchangeability on synthetic light curves and on 12 TESS stars.

A shuffled-block null keeps the structure inside a block and removes it between blocks, so the threshold guards against what survives that operation. Thompson et al. (2018) used inverted and scrambled light curves for Kepler and noted that inversion assumes false alarms that are symmetric under a flip of the flux. For TESS, Kunimoto et al. (2023a) found pileups at aliases of the 13.7-day orbit in the BLS detections of 92 899 M dwarfs and the same pileups in their inverted light curves, and Kunimoto et al. (2025) built their false alarms by scrambling alone. Because the SDE is a z-score over the trial periods searched, it changes with the period range, and a threshold applies only to the grid on which it was made.

3.5. *Hardware and software*

All runs used an AMD Ryzen 9 9950X3D 16-Core Processor with 32 logical processors and an AMD Radeon RX 7900 XTX (gfx1100) with ROCm 7.2.53211, under Python 3.12.13, torch 2.12.0+rocm7.2, NumPy 2.5.4, Numba 0.68.0, astropy 8.0.2 and TLS 2.0.1. The RX 7900 XTX is the consumer card of the Navi 31 die; the workstation card of the same die, the Radeon PRO W7900, has the same 96 compute units and boost clock of 2.50 GHz, with 48 GB of memory at 864 GB/s in place of 24 GB at 960 GB/s (GPU-Monkey 2026). The package has been run only with ROCm on this GPU, not with CUDA.

4. RESULTS

4.1. *Agreement of exact mode with TLS*

On each of the 14 characterization cases, exact mode agrees with TLS fused to within float64 rounding. The period grid, the duration grid, the number of transits and every integer field are identical, and the best period is the same grid point. Of the 41 result fields, 28 are identical in all cases; the other 13 differ in the last digits. Table 12 (Appendix A) lists the 10 fields with the largest differences. The χ^2 of every period differs by at most 1.8×10^{-11} relative and the SDE by less than 10^{-11} (Table 12; a rerun moves the last digit of these differences). Figure 5 draws the relative difference of the χ^2 at every trial period of one case, and Table 13 gives its largest value for each case. The largest relative difference, 10^{-7} in the raw power, arises because the power divides by the spread of χ^2, which amplifies its rounding by about 10^4. In the tests, each component is checked against a reference implementation, and the screened search is checked against a brute-force search over all trial epochs.

4.2. *Adversarial cases*

Eleven further light curves are built to stress the exact search, each compared with TLS fused: two planets of equal depth whose peaks nearly tie; the shortest templates of the grid (periods of 0.5 to 0.7 d at 10-minute cadence) and the longest (periods of 20 to 45 d over a baseline of 90 d); time stamps that are duplicated and time stamps on a regular lattice, which tie in phase; the same light curve in BTJD and in BJD (an offset of 2 459 000 d); errors that span five orders of magnitude and errors of 10^{-9} and 10^{3} among errors of 5×10^{-4}; a light curve of 40 points; and a light curve with 60% of its baseline in gaps. The period and duration grids are equal in all eleven. The χ^2 of every period agrees to at most 2.8×10^{-13} relative over the eleven (the tests bound it at 10^{-6}), and in ten of them, where the best period is the same, every other field of the results agrees to at most 4.5×10^{-9} relative. In the light curve of 40 points two trial periods have χ^2 that differ by 1.4×10^{-13} relative, and exact mode returns 1.4135 d where TLS fused returns 1.3898 d: the agreement to float64 rounding holds for the χ^2 of every period, and the best period can differ when two periods lie within rounding of each other. The BJD light curve gives the period of its BTJD twin to a relative 10^{-9} and not bit for bit, because the grid follows the span of the times, which BJD rounds at 10^{-10} d.

4.3. *The box search against a continuous search*

In the tests, the box search agrees with a plain NumPy loop over the algorithm of Sect. 2.3, and on the light curves of the tests and the 200 trial periods of Table 14 (Appendix A) it never scored above an exhaustive continuous box search over the same durations. Its power agreed bit for bit between a CPU and a GPU, across batch sizes and on repeated runs, except in one complete run of the test file, in which two batch sizes and a call with weights of one gave powers that differ by up to 1.3×10^{-9} relative, with equal depths: after eight variants of the scoring function torch stops compiling it and runs further variants eagerly, which differs from the compiled scoring in the last digits, and two calls of one test can fall on either side of that limit. Since version 0.2.0 the package raises that limit to 64 variants when it is lower; the complete run of the test file was not repeated to see whether the difference is gone. Its 32-bit scores give the same best period as 64-bit scores.

The box search differs from the continuous BLS statistic in three ways: it bins the phase, it quantizes the flux to 10^{-9}, and it scores in 32-bit floats. Phase binning has the largest effect, because box edges must lie on bin edges. Table 14 measures it on 200 trial periods of Gaussian noise (5 000 points over 400 d), for at least 3 or 12 bins across the shortest duration and duration steps of 1.5 or 1.1. We rescore the box that the binned search finds without bins and divide by the best statistic over every trial epoch at a point and every duration on a grid of ratio 1.02. At the defaults of 12 bins and a step of 1.1, the box keeps a median of 0.93 of the continuous statistic, so it loses 7%; at 3 bins and a step of 1.5 it keeps 0.80.

4.4. *Recovery of injected planets at a matched false-alarm rate*

Each search was run on 1 200 light curves with an injected planet in each of three synthetic classes of one sector (27.4 d at 2-minute cadence), and on 1 000 light curves with nothing injected (`validation/run.py`). The classes are white noise of 1 000 ppm, red noise (700 ppm of white noise and an Ornstein–Uhlenbeck process of 500 ppm with a correlation time of 0.3 d) and a variable star (700 ppm of white noise and a rotation signal: a sinusoid of 3 000 ppm at a period drawn log-uniformly from 0.8 to 10 d, with its first harmonic at 40 percent of the amplitude, at random phases). The red and variable light curves are divided by a biweight filter of 3 d (wotan) after the planet is injected, the same filter for every search; white noise is not filtered. The planets have periods of 0.8, 1.6, 3.2, 6.4 and 12.8 d, six target signal-to-noise ratios from 4 to 20 (against the noise at the transit duration for the red and variable classes;

Pont et al. 2006), an impact parameter uniform from 0 to 0.9 and a random epoch. A search finds a planet when its best peak has a period within 1% of the planet's and a transit time within half a duration of a transit, and an SDE at or above the threshold of its own SDE statistic: the 95th percentile of the SDE of the 1 000 noise-only light curves of the same class. Peaks at a harmonic of the period are counted apart and are not found. The hybrid is run with 1, 5, 10 and 20 refined peaks, and it is counted as finding a planet when any of those peaks is at the planet's period and the SDE of the box spectrum is above the threshold: its share is that of a candidate list of k peaks, not of one best peak, which is why it is never below the box search and can exceed exact mode. By the protocol of the validation, a hybrid whose paired lower bound falls more than 10 points below exact mode in any class is reported as candidate generation and not as a search, which its loss in the variable class makes it. Because the threshold is set for each search and class, the searches are compared at a false-alarm rate of 5%, not at one SDE.

Figure 2 gives the share found at each signal to noise, and Table 2 the thresholds and the shares with their Wilson intervals. In white noise the box search differs from exact mode by +0.1 points (−0.6 to +0.9, the score interval of Tango 1998 for paired shares, S/N of 8 and above), binned mode by −0.1 (−0.7 to +0.4) and the hybrid with 5 peaks by +0.6 (+0.1 to +1.5). In red noise binned mode differs by +0.1 (−0.4 to +0.7), the box search by −6.1 (−8.0 to −4.7) and the hybrid with 5 peaks by −5.1 (−7.1 to −3.5). In the variable class the box search differs by −31.0 (−34.3 to −27.9), the hybrid with 5 peaks by −30.9 (−34.2 to −27.7) and binned mode by −1.4 (−2.5 to −0.6). Refining 20 peaks in place of 5 changes the share found by the hybrid by at most 0.3 points in any class, pooled over the signal-to-noise ratios.

The loss of the box search in the variable class is that of its statistic, not of this implementation. On the 200 injected light curves of that class at a signal to noise of 8 and their 200 nulls (`validation/box_variable.py`), each search again at the 95th percentile of its own nulls, astropy's `BoxLeastSquares` with the same periods and durations finds 5 planets (2.5%, 1.1 to 5.7) and the box search 7 (3.5%), a paired difference of −1.0 points (−3.6 to +0.9); exact mode finds 99 (49.5%). The thresholds here come from 200 nulls, so these counts differ by one or two from those of Table 2, whose thresholds come from 1 000. The box search's SDE is a plain z-score over the trial periods, and the spectrum of a light curve with a rotation signal carries a trend across them. With the running-median detrending that TLS applies to its own spectrum applied to the box spectrum, the same box statistic finds 77 (38.5%, 32.0 to 45.4), +35.0 points more than the plain box (+28.7 to +41.8) and −11.0 points fewer than exact mode (−16.7 to −6.1). The detrending is not part of the box search as released, because its effect on the false-alarm rate of the per-star threshold of Sect. 3.4, which the shuffles set on the undetrended spectrum, was not measured; the difference that remains between the detrended box and exact mode is not explained here. The loss in red noise was not tested the same way.

On the 12 real TESS stars of Sect. 3.3, with the threshold of each search at the 95th percentile of its own 1 212 nulls, exact mode finds 82% of the 180 injected planets at a signal to noise of 8 (75 to 87) and 98% at 10 (94 to 99), and every search finds all of them at 14 and 20. The other three searches find shares within 1 point of exact mode's: against it, binned mode differs by −0.3 points (−1.0 to +0.3), the box search by +0.7 (−0.5 to +2.0) and the hybrid with 5 peaks by +1.0 (−0.3 to +2.4), each interval holding zero. These stars were chosen quiet, so the loss of the box search and the hybrid on the rotating star of the synthetic classes does not appear on them; Table 2 gives their thresholds and shares.

Table 3 gives the share found at each injected period, over every signal to noise from 4 to 20 of that period (240 planets per period in a synthetic class, 216 on the real stars). The largest difference between binned mode and exact mode in any cell is in the variable class at 3.2 d, 55.0 against 57.1%; at 12.8 d, where a sector holds one or two transits, binned mode finds 72.1 against 72.1% in white noise and 62.0 against 62.5% on the real stars. The box search loses nothing at 12.8 d either (74.2% in white noise, 66.2% on the real stars). Its loss is in the variable class at every period (24.2 against 43.3% at 0.8 d, 37.5 against 61.7% at 12.8 d) and in red noise at short periods (49.2 against 62.9% at 0.8 d, 70.4 against 71.2% at 12.8 d). The share found falls with period in white noise and on the real stars and rises with period in red noise and in the variable class; the cause of either shape is not measured here.

The comparison above is at one false-alarm rate. Figure 3 gives, for each class and for the real stars, the share of the injected planets found against the share of the noise-only light curves above the same SDE threshold, over every threshold (the true-positive rate against the false-positive rate, as Hu et al. 2026 draw it for GTLS, TLS and BLS), with the planets at a signal to noise of 8 and above. At a false-positive rate of 1%, exact mode finds 97.6% in white noise, 93.5% in red noise, 69.4% in the variable class and 85.6% on the real stars, binned mode 97.8, 93.6, 70.5 and 86.2%, and the box search 98.2, 85.2, 41.0 and 89.2%. In white noise the four searches stay within about one point of each other at every rate from 1% to 50%. In red noise binned mode stays within one point of exact mode at every

rate, while the box search and the hybrid fall further behind exact mode as the rate falls (85.2 against 93.5% at 1%, 89.0 against 95.1% at 5%), so the comparison at 5% understates their loss at a stricter rate. In the variable class the box search and the hybrid stay far below exact mode at every rate below 80%, and the curves of binned and exact mode cross between 2% and 2.5%: binned mode is above exact mode at 1% (70.5 against 69.4%) and below it at 5%. On the real stars the four searches are within a point of each other at 5% and above, and at 1% the box search and the hybrid are above exact mode (89.2 against 85.6%); below 1% the threshold of 1 212 nulls is set by a handful of them and the curves are not resolved. The order of the searches at 5% therefore holds at 1% in white and red noise and not in the variable class or on the real stars; why the box search leads on the real stars at 1% is not measured here.

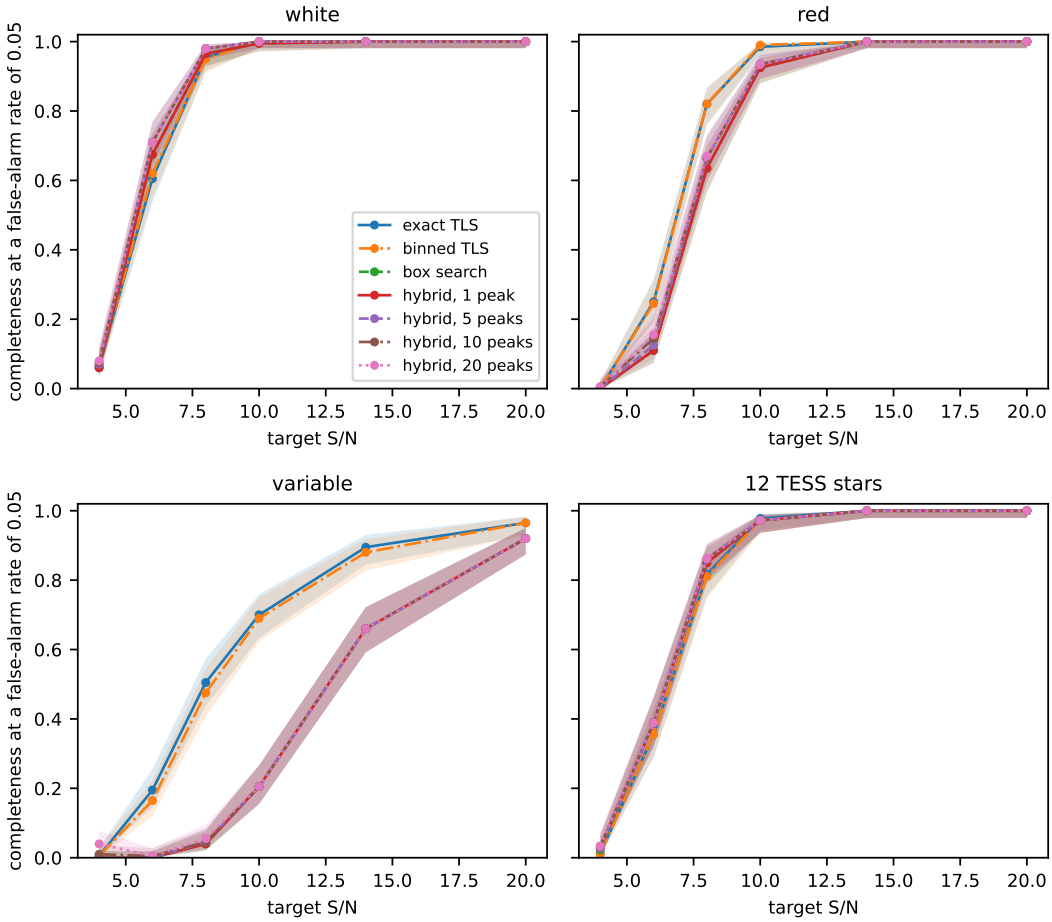


Figure 2. Share of the injected planets found at a false-alarm rate of 5%, against the target signal to noise, in each noise class and on the 12 real TESS stars: every search, with its 95% Wilson band. The bands of the exact and binned modes overlap.

Table 2. Threshold of each search at a false-alarm rate of 5% (SDE, with a bootstrap interval) and the share of injected planets found at target signal to noise of 6, 8 and 10, with the 95% Wilson interval; 200 planets in each cell of the synthetic classes and 180 in each cell of the real stars.

white noise

search	threshold	S/N 6	S/N 8	S/N 10
exact	7.5 (7.3 to 7.8)	60% (54 to 67)	96% (92 to 98)	100% (98 to 100)
binned	7.6 (7.4 to 7.7)	62% (55 to 68)	95% (91 to 97)	100% (98 to 100)
box	6.4 (6.3 to 6.6)	68% (61 to 74)	96% (93 to 98)	100% (97 to 100)
hybrid1	6.4 (6.3 to 6.6)	68% (61 to 74)	96% (93 to 98)	100% (97 to 100)
hybrid5	6.4 (6.3 to 6.6)	71% (64 to 77)	98% (95 to 99)	100% (98 to 100)
hybrid10	6.4 (6.3 to 6.6)	71% (64 to 77)	98% (95 to 99)	100% (98 to 100)
hybrid20	6.4 (6.3 to 6.6)	71% (64 to 77)	98% (95 to 99)	100% (98 to 100)

red noise

search	threshold	S/N 6	S/N 8	S/N 10
exact	7.5 (7.3 to 7.6)	25% (20 to 31)	82% (76 to 87)	98% (96 to 99)
binned	7.5 (7.3 to 7.7)	24% (19 to 31)	82% (76 to 87)	99% (96 to 100)
box	5.2 (5.1 to 5.2)	11% (7 to 16)	64% (57 to 70)	92% (88 to 95)
hybrid1	5.2 (5.1 to 5.2)	11% (7 to 16)	64% (57 to 70)	92% (88 to 95)
hybrid5	5.2 (5.1 to 5.2)	12% (9 to 18)	66% (60 to 73)	94% (89 to 96)
hybrid10	5.2 (5.1 to 5.2)	14% (10 to 20)	66% (60 to 73)	94% (89 to 96)
hybrid20	5.2 (5.1 to 5.2)	16% (11 to 21)	67% (60 to 73)	94% (89 to 96)

variable star

search	threshold	S/N 6	S/N 8	S/N 10
exact	9.2 (9.0 to 9.4)	20% (15 to 26)	50% (44 to 57)	70% (63 to 76)
binned	9.2 (9.0 to 9.4)	16% (12 to 22)	48% (41 to 54)	69% (62 to 75)
box	7.8 (7.7 to 7.9)	0% (0 to 2)	4% (2 to 8)	20% (15 to 27)
hybrid1	7.8 (7.7 to 7.9)	0% (0 to 2)	4% (2 to 8)	20% (15 to 27)
hybrid5	7.8 (7.7 to 7.9)	0% (0 to 2)	4% (2 to 8)	20% (15 to 27)
hybrid10	7.8 (7.7 to 7.9)	0% (0 to 3)	4% (2 to 8)	20% (15 to 27)
hybrid20	7.8 (7.7 to 7.9)	0% (0 to 3)	6% (3 to 10)	20% (15 to 27)

12 TESS stars

search	threshold	S/N 6	S/N 8	S/N 10
exact	8.7 (8.4 to 9.1)	36% (29 to 43)	82% (75 to 87)	98% (94 to 99)
binned	8.8 (8.4 to 9.1)	36% (29 to 43)	81% (75 to 86)	97% (94 to 99)
box	6.8 (6.6 to 7.1)	39% (32 to 46)	85% (79 to 89)	97% (94 to 99)
hybrid1	6.8 (6.6 to 7.1)	39% (32 to 46)	85% (79 to 89)	97% (94 to 99)
hybrid5	6.8 (6.6 to 7.1)	39% (32 to 46)	86% (80 to 90)	97% (94 to 99)
hybrid10	6.8 (6.6 to 7.1)	39% (32 to 46)	86% (80 to 90)	97% (94 to 99)
hybrid20	6.8 (6.6 to 7.1)	39% (32 to 46)	86% (80 to 90)	97% (94 to 99)

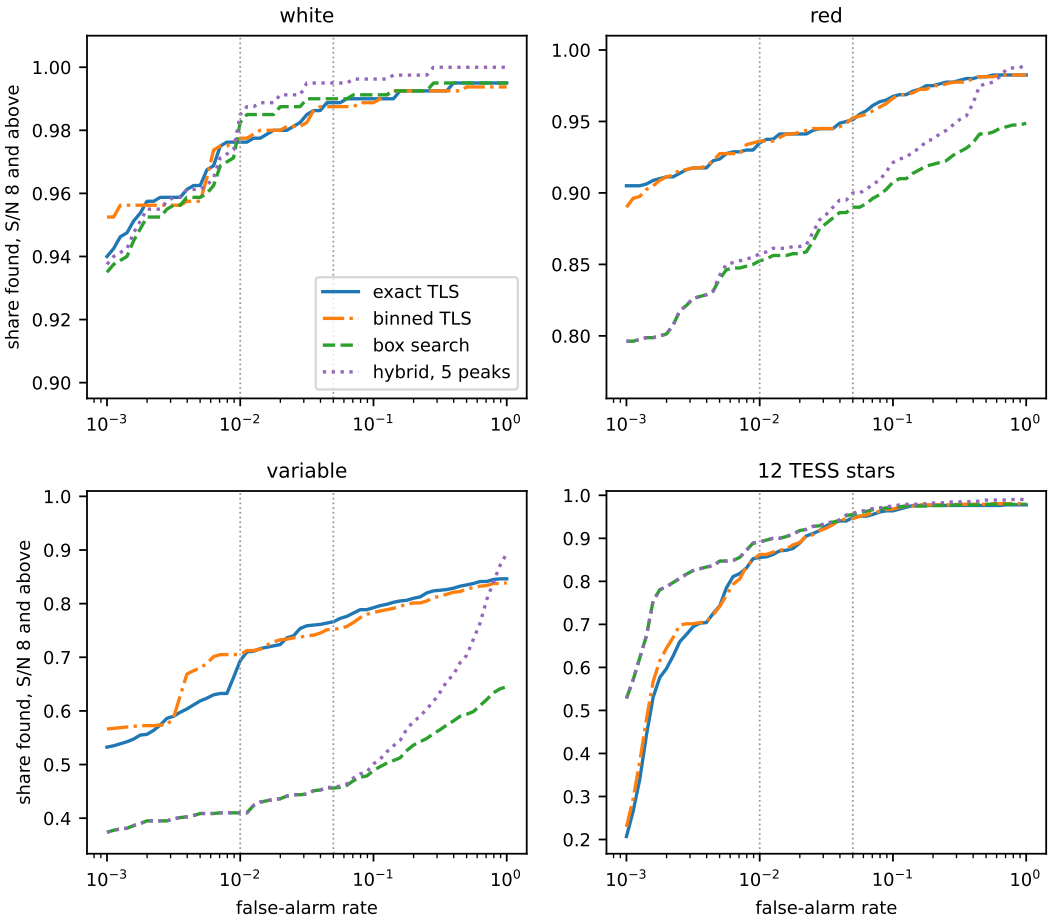


Figure 3. True-positive rate against false-positive rate of the four searches over every SDE threshold, in each noise class and on the 12 real TESS stars, for the planets injected at a signal to noise of 8 and above against the noise-only light curves of the same class. The point at a false-positive rate of 5% is the comparison of Table 2.

Table 3. Share of the injected planets found at a false-alarm rate of 5%, by injected period, over every signal to noise from 4 to 20 (240 planets per cell in the synthetic classes, 216 on the real stars), in percent with the 95% Wilson interval.

search	0.8 d	1.6 d	3.2 d	6.4 d	12.8 d
white noise					
exact	80.8 (75.4–85.3)	76.7 (70.9–81.6)	78.3 (72.7–83.1)	77.5 (71.8–82.3)	72.1 (66.1–77.4)
binned	80.8 (75.4–85.3)	76.7 (70.9–81.6)	78.3 (72.7–83.1)	78.3 (72.7–83.1)	72.1 (66.1–77.4)
box	80.4 (74.9–84.9)	77.9 (72.3–82.7)	79.2 (73.6–83.8)	79.6 (74.0–84.2)	74.2 (68.3–79.3)
hybrid, 20 peaks	82.5 (77.2–86.8)	79.6 (74.0–84.2)	79.6 (74.0–84.2)	80.0 (74.5–84.6)	75.8 (70.0–80.8)
red noise					
exact	62.9 (56.6–68.8)	66.7 (60.5–72.3)	67.1 (60.9–72.7)	70.0 (63.9–75.4)	71.2 (65.2–76.6)
binned	63.3 (57.1–69.2)	66.7 (60.5–72.3)	67.1 (60.9–72.7)	69.6 (63.5–75.1)	71.2 (65.2–76.6)
box	49.2 (42.9–55.5)	57.9 (51.6–64.0)	62.9 (56.6–68.8)	65.4 (59.2–71.1)	70.4 (64.4–75.8)
hybrid, 20 peaks	52.1 (45.8–58.3)	59.2 (52.9–65.2)	64.6 (58.3–70.4)	66.7 (60.5–72.3)	71.2 (65.2–76.6)
variable star					
exact	43.3 (37.2–49.7)	52.5 (46.2–58.7)	57.1 (50.8–63.2)	57.5 (51.2–63.6)	61.7 (55.4–67.6)
binned	42.1 (36.0–48.4)	51.2 (45.0–57.5)	55.0 (48.7–61.2)	55.8 (49.5–62.0)	60.8 (54.5–66.8)
box	24.2 (19.2–30.0)	22.9 (18.1–28.6)	32.1 (26.5–38.2)	35.4 (29.6–41.7)	37.5 (31.6–43.8)
hybrid, 20 peaks	25.8 (20.7–31.7)	23.8 (18.8–29.5)	32.9 (27.3–39.1)	35.8 (30.0–42.1)	38.8 (32.8–45.0)
real stars					
exact	71.3 (64.9–76.9)	72.2 (65.9–77.8)	72.2 (65.9–77.8)	68.5 (62.0–74.3)	62.5 (55.9–68.7)
binned	71.3 (64.9–76.9)	71.8 (65.4–77.3)	72.2 (65.9–77.8)	68.5 (62.0–74.3)	62.0 (55.4–68.2)
box	72.7 (66.4–78.2)	72.7 (66.4–78.2)	72.7 (66.4–78.2)	68.5 (62.0–74.3)	66.2 (59.7–72.2)
hybrid, 20 peaks	73.1 (66.9–78.6)	73.1 (66.9–78.6)	72.7 (66.4–78.2)	69.0 (62.5–74.8)	66.7 (60.1–72.6)

4.5. *Calibration of the shuffle threshold*

The threshold of Sect. 3.4 was tested on synthetic light curves of one flattened sector, 19 740 points at 2-minute cadence, from the generator of the package (`validation/threshold.py`): 2 000 noise-only light curves in each of three classes, searched with the box search from 0.5 to 14 d together with 48 one-day block shuffles of each. The white class has white noise. The red class adds a correlated process to white noise, and the variable class adds a rotation signal. A second threshold comes from the same 48 shuffles of the inverted light curve (the flux reflected about its median). Table 4 gives the share of light curves whose best peak exceeds the threshold. Figure 6 (Appendix A) gives the distribution of the SDE of the noise-only light curves of each search in each class and on the real stars, with the thresholds at a false-alarm rate of 5% of Table 2 and the 48-shuffle thresholds of the box search beside them: in white and red noise the shuffle threshold of most light curves lies above the box search's 5% threshold; in the variable class every shuffle threshold lies below it, which is the crossing of two thirds of the light curves in Table 4; on the real stars about half the stars' shuffle thresholds lie below it.

Table 4. Share of noise-only light curves whose best peak exceeds the largest SDE of their 48 shuffles (the inverted light curve for the last column), with the 95% Wilson interval in parentheses, against the nominal 1/49.

class	curves	above	shuffle threshold	nominal	inverted threshold
white	2 000	50	2.5% (1.9 to 3.3)	2.0%	2.7% (2.1 to 3.5)
red	2 000	16	0.8% (0.5 to 1.3)	2.0%	1.5% (1.0 to 2.1)
variable	2 000	1 325	66.2% (64.1 to 68.3)	2.0%	62.7% (60.6 to 64.8)

For white noise the share is 2.5% (1.9 to 3.3), which contains $1/49 = 2.04\%$. For red noise it is 0.8% (0.5 to 1.3): the threshold is conservative. We have not measured why; a candidate is the steps that the one-day blocks leave at their joints in flattened red noise (argued, not measured). For the variable class 66.2% of the light curves cross. Of the 1 325 crossings, 1 264 (95.4%, 94.1 to 96.4) have their best period at the rotation period or at its ratios 1/3, 1/2, 2 and 3, within 1%. A shuffle in blocks of one day destroys periodicity, so this null treats a real periodic signal as a signal, and the share measures the strength of the rotation and not a false-alarm rate. The 61 crossings left (3.05% of the light curves, 2.4 to 3.9) are above 1/49, and we do not call them noise. Of the 675 variable light curves that do not cross, 64.3% also peak at the rotation.

On the 12 real stars of Sect. 3.3, each searched with 199 shuffles, the test is whether the share of the shuffles that the star's own best peak outranks is uniform between 0 and 1, as exchangeability requires. The mean share is 0.61, and a Kolmogorov–Smirnov test against the uniform distribution gives 0.31 with $p = 0.16$, so exchangeability is not rejected on these stars; with 12 stars the test has little power, and the result does not make the threshold calibrated on real stars. Eight further stars of an earlier selection by the same rule, none in the list of Sect. 3.3, give a mean share of 0.677 and $p = 0.12$; pooled, the 20 stars give a mean of 0.64 and $p = 0.1$.

Planets were injected into the first light curves of each class, so that the same noise is searched with and without a planet (250 planets at each of six signal-to-noise ratios per class, the signal to noise of the red and variable classes taken against the noise at the transit duration; Pont et al. 2006). A planet is found when its best peak is at its period, within 1% and half a duration of a transit, and above the largest SDE of its own 48 shuffles. Table 5 gives the shares: 87 to 92% of the white-noise planets at a signal to noise of 10 to 20, 89 to 90% of the red-noise planets at 14 to 20 and 84% of the variable-class planets at 20. The best peak is at the injected period in 250 of 250 white-noise planets at each signal to noise from 10 to 20 and in 250 of 250 red-noise planets at 14 and 20 (247 of 250 at 10), and the planet raises its own threshold by a median factor of 1.3 to 1.6 at 14 to 20. The threshold of the inverted light curves finds 98.8 to 100% of the white-noise planets at 10 to 20, 100% of the red-noise planets at 14 and 20 (88.0% at 10), and 83.2 to 96.8% of the variable-class planets at 14 and 20.

Table 5. Injected planets: the planets (250 in each row), those whose best peak is at the injected period (within 1%), those at a harmonic, and the share found above the threshold of their own shuffles and of their inverted shuffles, with the 95% Wilson interval.

class	S/N	planets	period	harmonic	above shuffle threshold	above inverted threshold
white	4	250	57	19	2.4% (1.1 to 5.1)	2.8% (1.4 to 5.7)
white	6	250	192	17	44.4% (38.4 to 50.6)	48.0% (41.9 to 54.2)
white	8	250	247	3	82.8% (77.6 to 87.0)	94.0% (90.3 to 96.3)
white	10	250	250	0	86.8% (82.0 to 90.4)	98.8% (96.5 to 99.6)
white	14	250	250	0	88.4% (83.8 to 91.8)	100.0% (98.5 to 100.0)
white	20	250	250	0	91.6% (87.5 to 94.4)	100.0% (98.5 to 100.0)
red	4	250	7	10	0.4% (0.1 to 2.2)	0.0% (0.0 to 1.5)
red	6	250	87	9	4.0% (2.2 to 7.2)	7.2% (4.6 to 11.1)
red	8	250	211	3	45.2% (39.1 to 51.4)	59.6% (53.4 to 65.5)
red	10	250	247	0	73.6% (67.8 to 78.7)	88.0% (83.4 to 91.5)
red	14	250	250	0	89.2% (84.7 to 92.5)	100.0% (98.5 to 100.0)
red	20	250	250	0	89.6% (85.2 to 92.8)	100.0% (98.5 to 100.0)
variable	4	250	1	6	0.4% (0.1 to 2.2)	0.4% (0.1 to 2.2)
variable	6	250	17	8	0.4% (0.1 to 2.2)	4.0% (2.2 to 7.2)
variable	8	250	73	10	11.2% (7.9 to 15.7)	19.2% (14.8 to 24.5)
variable	10	250	151	7	31.6% (26.2 to 37.6)	52.8% (46.6 to 58.9)
variable	14	250	217	0	65.6% (59.5 to 71.2)	83.2% (78.1 to 87.3)
variable	20	250	244	2	83.6% (78.5 to 87.7)	96.8% (93.8 to 98.4)

4.6. *Run times*

Each timed call takes the times and fluxes on the host and returns its results on the host, so the GPU rows include the copies to and from the GPU and the final synchronization; Table 15 (Appendix A) lists what each row includes. The box search is timed with the period grid given and does not fit the best period; the TLS rows and the rows of `transitgpu.tls` time the whole `power()` call, grids, template cache, spectra and final fit included; the hybrid row times the box search and one such call per peak; for astropy's `BoxLeastSquares`, the object is built before the timer starts. A call on 4 000 points compiles the kernels before timing, and each time is the median of seven calls. That first call of a fresh process, on 4 000 points, which the tables exclude, takes 2.10 s for the box search, 1.25 s for exact mode, 1.05 s for binned mode and 2.60 s for the hybrid when the compiled kernels are on disk from an earlier process (the median of five fresh processes); with those caches cleared, the call compiles them (one run) and takes 21.4 s for the box search, 2.20 s for exact mode and 16.9 s for the search of many light curves; the box search and the search of many light curves build their HIP kernel with hipcc.

The rows of the device and the rows of TLS fused on 16 threads (`tls16`, `tls16-exact`) were timed on one machine with one set of software versions, in one session on 2026-11-05, each as the median of seven calls, every part pinned to cores 0–15 on an otherwise idle machine with nothing else on the GPU. The rows that time only the CPU (astropy, TLS on one thread, exact mode of TLS on one thread, TLS on 16 threads over the 64 light curves, and exact mode of TLS on 16 threads at the input of the GTLS paper) are those of an earlier session on the same machine, 2026-10-26, which used astropy 8.0.1 where the session of 2026-11-05 used 8.0.2; no code of the package runs in them. A run of each of them at 2×10^4 points on 2026-11-05 took 0.98 to 1.01 times the time of the earlier session. Table 6 gives the times.

At 2×10^5 points, exact mode takes 2.8 times less time than TLS fused on 16 threads and 28 times less than on one thread. It takes about as long as TLS fused-pl on 16 threads (9.91 against 10.8 s), whose statistic is approximate. The ratio of TLS fused on 16 threads to exact mode is 2.5, 2.3 and 2.8 at the three sizes of Table 6, and 2.1 and 2.3 on the two longer light curves of Sect. 5.3, so it does not grow with the number of points. Binned mode approximates the statistic of TLS fused. It takes 19 times less time than TLS fused on 16 threads and 6.7 times less than exact mode. It also takes 7.3 times less than TLS fused-pl on 16 threads, which is the more accurate approximation of the two (Sect. 2.6), so that ratio compares speed only. The box search takes 145 times less than astropy's `BoxLeastSquares` on one thread, which uses a different list of durations, and 23 times less than TLS fused-pl on 16 threads; the box search and TLS compute different statistics, so that ratio is not the speed-up of one algorithm.

Table 6. Time in seconds of one search over the period grid of TLS (91 607 trial periods from 0.5 to 14 days), as the median of seven calls. The box searches score a box and the others a template.

search	2×10^4 points	10^5 points	2×10^5 points
transitgpu box search, GPU	0.223	0.333	0.477
`transitgpu.tls` exact mode, GPU	2.57	5.33	9.91
`transitgpu.tls` binned mode, GPU	1.11	1.37	1.48
`transitgpu.tls` piecewise-linear mode (gpu-pl), GPU	1.24	4.08	6.77
hybrid search, 5 peaks, 1% window, GPU	0.407	0.697	1.06
astropy `BoxLeastSquares`, 1 thread	37.2	51.3	69.1
TLS fused-pl, 16 threads	1.47	4.09	10.8
TLS fused, 16 threads	6.54	12.5	27.7
TLS fused-pl, 1 thread	20.2	49.1	85.6
TLS fused, 1 thread	91.0	166	277

The box search of 64 light curves of 2×10^4 points takes 5.24 s in one call; TLS fused-pl on 16 threads takes 92.4 s for the same light curves one after another. A threshold made from 48 shuffled copies of one light curve (Sect. 3.4), the 48 copies searched in one call (48×91 607 trial periods; the search of the light curve itself is a further search), takes 3.96, 6.19 and 9.79 s at the three sizes. With periods up to half the baseline, the box search takes 0.690 s for 72 000 points over 1 500 days (191 837 trial periods) and 1.93 s for 144 000 points over 3 000 days (391 781 trial periods).

4.7. *The biweight detrend*

Table 7 times one detrend of a synthetic sector of 27.4 days, with its transits masked, at the cadences of the full-frame images (30 and 10 minutes and 200 seconds) and of the 2-minute and 20-second targets, against wotan 1.10 on one CPU thread (`benchmarks/bench_detrend.py`). Each time is the median of seven calls after a first call, from the arrays on the host to the trend on the host, so the GPU rows include building the windows and both transfers. The kernel takes 3.59 ms for a 2-minute sector with a window of 0.75 d, 61 times less than wotan, and 8.58 ms with a window of 1.5 d, 47 times less. It is faster than the torch version on the same GPU at every padded width it is run at, from 64 to 2 048 points. Its first call compiles it when the Triton cache does not hold that width, which at 2 048 points from an empty cache took 34.3 s. The kernel is not run above 2 048 points, so windows padded wider than 2 048 points run the torch version on the GPU: at 20-second cadence it takes 612 ms against 6 855 ms for wotan. At 30-minute cadence the torch version on the GPU takes 2.36 ms, with single calls from 1.57 to 2.51 ms, against 1.97 ms for wotan: it makes 220 kernel launches a call at a width of 64 points and 178 at 128, and at the narrowest windows that fixed cost is about the time of wotan's whole call. The kernel serves that width by default, in 0.249 ms. Every row agrees with wotan's trend to 2.2×10^{-16} relative, with the NaN values at the same positions.

Table 7. Time in milliseconds of one biweight detrend of a sector of 27.4 days, with the transits masked, by cadence and window, with the padded width of the windows; the median of seven calls after the first. The torch version on the CPU ran with torch's default of 16 threads on this machine. At 20 s it was not run, and the kernel is not run above its width limit.

cadence	30 min	10 min	200 s	2 min	2 min	20 s
window (d)	0.75	0.75	0.75	0.75	1.5	0.75
padded width	64	128	512	1 024	2 048	4 096
wotan 1.10, CPU, 1 thread	1.97	10.8	80.4	219	400	6 855
torch version, CPU, 16 threads	1.96	6.21	61.9	353	766	–
Triton kernel, GPU	0.249	0.487	2.39	3.59	8.58	–
torch version, GPU	2.36	2.50	7.73	13.7	37.3	612

4.8. *Where the time goes*

The box search touches each point once per period and then scores a few hundred bins, so its time grows more slowly than the number of points: 0.223 s at 2×10^4 points and 0.477 s at 2×10^5. TLS fused-pl on 16 threads grows from 1.47 s to 10.8 s over the same range, so the ratio of the two grows from 6.6 to 23.

Exact mode sorts the points of each period and screens the trial epochs. Table 16 (Appendix A) counts the trial epochs that survive each cascade bound for one batch of 64 periods in each of four period ranges at each size, as a share of those tested at the durations each period allows. Between 0.04% and 0.47% of trial epochs reach the exact correlation in every range and size, or 56 to 164 per period; U_2 removes most of what U_1 leaves, and U_3 most of what U_2 leaves. U_1 is most effective at long periods and small sizes: at 2×10^4 points and 5 to 14 d it leaves 64% of trial epochs, while at 2×10^5 points and 0.5 to 2 d it leaves all of them. Counted by template length, the share that reaches the correlation exceeds 1% only for the longest templates at each size, up to 2.77% for templates of 2 048 to 4 095 samples at 2×10^4 points and 2 to 5 d; these templates have both the most survivors and the most samples to sum.

Table 17 (Appendix A) splits the GPU time of exact mode over the whole grid by stage, from the profiler, with each kernel counted once. The fold, which sorts the points, grows with the number of points (0.38 to 2.59 s), and so does the screening (1.30 to 2.55 s), although the number of trial epochs tested in a batch of 64 periods barely changes with size (2.4×10^6, 2.3×10^6 and 2.3×10^6 at 0.5 to 1 d); we have not measured what causes that growth. The screening takes most of the time at 2×10^4 points, while at 2×10^5 the fold, the scans and the screening take similar shares. The profiler slows the run, and the host launches kernels while the GPU works, so these GPU times are not wall-clock times. Figure 7 (Appendix A) draws both tables.

4.9. *Sanity check on WASP-18 b*

This check shows that the searches run end to end on a real light curve with a known deep planet; it does not measure detection performance. We searched the first 13 days of the TESS 2-minute light curve of WASP-18 (TIC 100100827, sector 2; Figure 4). The light curve is the Pre-search Data Conditioning Simple Aperture Photometry (PDCSAP) flux of the Science Processing Operations Center (SPOC; Jenkins et al. 2016), flattened with wotan (Hippke et al. 2019) using a biweight filter with a 0.75 d window. Between 0.5 and 5 days, the box search finds a period of 0.94176 d, a duration of 1.90 h, a depth of 10 086 ppm and an SDE of 10.6. Exact mode finds the same period, a duration of 2.09 h, a depth of 10 915 ppm and an SDE of 25.9. The NASA Exoplanet Archive (Akeson et al. 2013) lists a period of 0.94145 d for WASP-18 b (Hellier et al. 2009). The two SDEs are on different scales: that of the box search is a z-score of its statistic without detrending, and that of exact mode is a z-score of the TLS statistic after running-median detrending, so their ratio is not a ratio of significance.

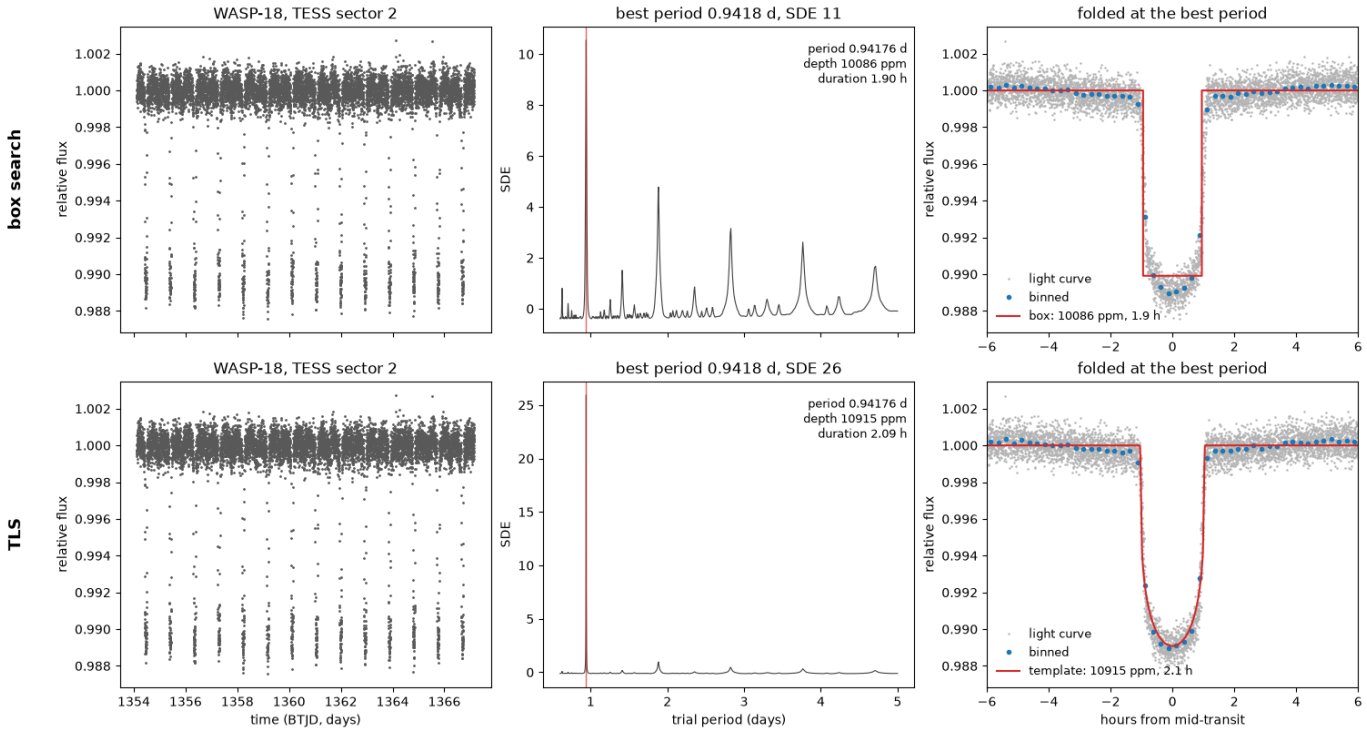


Figure 4. WASP-18 b in the first 13 days of TESS sector 2. Top: the box search; bottom: exact mode. Left: the flattened light curve. Middle: the SDE spectrum, with the best period marked. Right: the light curve folded at the best period, binned (blue), with the box or the template that was found (red).

5. COMPARISON WITH GTLS

5.1. *Design*

GTLS (Hu et al. 2026) is the other port of TLS to a GPU. It is written with CuPy and CUDA kernels and needs an NVIDIA device. This section describes the paper of Hu et al. (2026) and the source of release 0.5.1 of its package (commit 74e449c of its public repository, 21 January 2026). Sect. 2.4 follows the data layout of release 0.4.4, every function of which exists in 0.5.1, three of them changed in the memory they reuse between batches of periods and one in a prefix sum of the squared residuals; the archive of 0.4.4 does not hold the CUDA kernel file, so the kernels were read in 0.5.1 only.

Table 8 sets the steps of the two ports against TLS. Both keep the period grid of Ofir (2014) as TLS builds it. The source of GTLS release 0.5.1 has the three duration constants of TLS 2.0.1 (stellar radii from 0.13 to 3.5 solar radii and a transit of at most 0.12 of the period: `constants.py` and the kernel `durationsGrid` in `GPUFun.cu`, against `tls_constants.py`), while the paper of GTLS describes wider limits, $T_{14}P^{-2/3}$ from 20.05 to 620.84 $\mathrm{s}^{2/3}$ instead of the 41.39 to 546.94 $\mathrm{s}^{2/3}$ that it derives for TLS, and a transit of at most 0.15 of the period (Hu et al. 2026, Sect. 2.2.2); we did not find these limits in the source that we read. GTLS folds in float64 and sorts with CuPy; the sort is not described as a stable sort of the float64 phases. Its search of the trial epochs T_0 has two stages (Hu et al. 2026, Sect. 2.5): a shift every 0.125 of the template width at every period (`T0_FIT_MARGIN` in `constants.py`), then every shift around the roughly 200 periods with the highest SDE. TLS 2.0.1 tests a shift every 0.01 of the width at every period for templates wider than 100 cadences, and exact mode of `transitgpu.tls` tests the same shifts. GTLS computes the χ^2 outside the transit by a recursion over T_0 and, as its authors write, approximates it by subtracting the mean flux in the transit from unity and squaring the result (Hu et al. 2026, Sect. 2.3.4); the term inside the transit is a direct sum. In its source the times, the periods and the fold are float64, and the flux, the errors, the sums of the χ^2 and the best χ^2 of each period are float32 (`core.py`; the kernels `foldFast`, `calcAllFullSum` and `calcAllOutOfTransitResiduals` in `GPUFun.cu`). The exact backend of TLS 2.0.1 works in float64, its float32 storage belonging to its approximate backends, and exact mode of `transitgpu.tls` works in float64 throughout. We do not measure here what the difference does to the result of GTLS or to its speed.

Table 8. The steps of TLS 2.0.1, of GTLS (paper of Hu et al. 2026 and release 0.5.1) and of exact mode of `transitgpu.tls`.

	TLS 2.0.1	GTLS	`transitgpu.tls`, exact mode
period grid	Ofir (2014)	the same procedure, longest period half the span	TLS code
duration grid	stellar radii 0.13 to 3.5, at most 0.12 of the period	the same constants in the source; the paper describes 20.05 to 620.84 $s^{2/3}$ and at most 0.15	TLS code
fold and sort	float64 phase, sort by phase	float64 phase $t/P - \lfloor t/P \rfloor$, CuPy sort	the permutation of a stable float64 sort
trial epochs	every 0.01 of the width for templates wider than 100 cadences	stage 1: every 0.125 of the width; stage 2: every shift, for about 200 periods	those that TLS tests
χ^2	exact	outside the transit approximated, inside a direct sum	exact for the trial epochs that pass the bounds
precision	float64	times, periods and fold float64; flux, errors, χ^2 sums and best χ^2 float32	float64 throughout
SDE	from χ^2, detrended	from χ^2, detrended (Hu et al. 2026, Sect. 2.4)	TLS code

5.2. *Exactness*

Exact mode reproduces the χ^2 of TLS fused at every trial period: it agrees to at most 1.8×10^{-11} relative on the 14 characterization cases of TLS and to at most 2.8×10^{-13} on the adversarial cases (Sects. 4.1 and 4.2). Its grids, statistics and final fit are those of TLS. GTLS departs from the χ^2 of TLS in three ways that its authors state or that its source shows: the first stage tests a coarser set of trial epochs, the arrays are float32, and the term outside the transit is approximated. Its search space may also differ: the paper describes a wider range of durations than that of TLS, which the source that we read does not show. A χ^2 taken over a subset of the trial epochs of the same durations is not lower than the minimum over all of them, so the χ^2 of GTLS at a period that is not among the refined candidates is a minimum over fewer shifts than that of TLS.

The authors report, on synthetic light curves of four years at 30-minute cadence with 110 ppm of white noise and injected Earth-sized transits, a precision of 9.3% and a recall of 79.4% for GTLS and 9.4% and 81.1% for TLS, with the thresholds SDE = 7 and SNR = 5, and call them statistically consistent (Hu et al. 2026, Table 1). If both searches ran on one set of light curves with these thresholds, an exact port gives the SDE and the SNR of TLS, so it makes the same decisions on every light curve and has the same precision and the same recall. A recall 1.7 points lower therefore means that the statistic of GTLS differs from that of TLS on some of the light curves. It does not say which difference causes this (the coarse trial epochs, the float32 arrays, the approximation or, if the paper's wider range of durations is used, the durations), and it does not say that GTLS is worse. No run of GTLS was obtained here (Sect. 5.3), so the comparison of this section is one of design.

5.3. *Run times*

On light curves of the size of those of the GTLS paper (30-minute cadence, 1 500 and 3 000 days, 191 837 and 391 781 trial periods from the grid of TLS to half the baseline), TLS 2.0.1 on 16 threads takes 37.2 s and 128 s, exact mode of `transitgpu.tls` 18.0 s and 56.0 s, and binned mode 6.05 s and 14.3 s. These rows were timed in the session of Table 6 (the rows of TLS on 16 threads, in the session of 2026-10-26), on the machine of Sect. 3.5, with float64 and the margin of T_0 of 0.01 of TLS. At the margin of 0.125 of GTLS, TLS on 16 threads takes 9.11 s and 44.2 s, and exact mode 7.57 s and 25.5 s (Table 10).

The paper of GTLS times its search on light curves made with batman at 30-minute cadence over about 1 500 and 3 000 days, on an NVIDIA RTX 4090, and TLS on an AMD Ryzen 9 7950X: 33.3 s for GTLS against 522 s for TLS at 1 500 days, and 138 s against 3 289 s at 3 000 days, with 79 s on two GPUs (Hu et al. 2026, Sect. 3.4). That section

does not state the noise, the planet, the thread count of TLS, the number of calls or the margin of T_0 of its timings; the default of its source is 0.125, and its Sect. 4.2 reports that lowering the margin from 0.1 to 0.01 raises the run time of GTLS by about 37% and that of TLS by about 8%.

The two GPUs are consumer cards of the same year, 2022, with the published and the attained figures of Table 9. The published peaks are the manufacturers' where their documents state them and a specification database's otherwise, as the table says; the attained column is what `benchmarks/peak.py` reaches on the GPU used here with a Triton kernel of independent fused multiply-adds and a copy of 2 GiB, a lower bound on the peak, measured with the device otherwise idle. Table 10 sets the published times of GTLS beside the times measured here on light curves of the same length and cadence. They come from two machines, two sets of light curves and two implementations that return different results, so the table gives no speed-up of one search on one machine. The TLS run of the GTLS paper took 57 and 74 times longer than TLS fused on 16 threads here at the margin of 0.125, which shows how far the two papers' baselines differ in hardware, version and settings before any search is compared.

Table 9. The GPU of the GTLS paper and the GPU used here. Published: the manufacturer's figure where its document states it (NVIDIA's Ada whitepaper, NVIDIA Corporation 2022, and product page, NVIDIA Corporation 2026; AMD's product support page, Advanced Micro Devices 2026, for the compute units and the clock), otherwise a specification database (Notebookcheck 2026), each named in the cell. Attained: `benchmarks/peak.py` on the GPU used here, at a shader clock of up to 2943 MHz. The FP64 figure of the RTX 4090 follows from the whitepaper's statement that its FP64 rate is 1/64 of its FP32 rate; the FP32 figure of the RX 7900 XTX counts dual-issue, and the FP64 figure of the database is that figure over 32.

	NVIDIA RTX 4090 (GTLS paper)	AMD RX 7900 XTX (this paper)
memory	24 GB GDDR6X, 384-bit (product page)	24 GB GDDR6, 384-bit (database)
cores and clock	16 384 CUDA cores, boost 2.52 GHz (product page)	96 compute units, boost 2.50 GHz (support page)
memory bandwidth, published	1 008 GB/s (whitepaper)	960 GB/s (database)
memory bandwidth, attained	not measured	767 GB/s (80% of the published)
FP32, published	82.6 TFLOPS (whitepaper)	61.4 TFLOPS with dual-issue (database)
FP32, attained	not measured	34.9 TFLOPS, single-issue
FP64, published	1.29 TFLOPS (1/64 of FP32)	1.9 TFLOPS (database, 61.4/32)
FP64, attained	not measured	1.12 TFLOPS (1/31 of the attained FP32)

Table 10. Run times on light curves of the length and cadence of the GTLS benchmark (30-minute cadence; 191 837 and 391 781 trial periods here), in seconds. Ours: the session of Table 6 (the TLS row, in the earlier session), the median of seven calls, float64, on the GPU of Table 9. GTLS: the times its paper publishes for its own light curves on an RTX 4090 (Hu et al. 2026, Sect. 3.4), with float32 sums, at a margin of T_0 the paper does not state (its source's default is 0.125); they come from another machine, other light curves and another statistic.

	TLS fused, 16 threads margin 0.01	exact mode margin 0.01	TLS fused, 16 threads margin 0.125	exact mode margin 0.125	GTLS, published
1 500 days	37.2	18.0	9.11	7.57	33.3
3 000 days	128	56.0	44.2	25.5	138

A run of GTLS release 0.4.4 through the ROCm build of CuPy on this GPU, with a module in the run script answering GTLS's three calls to NVIDIA's management library with the free memory that the HIP runtime reports, gave a finite χ^2 at fewer than 3% of the trial periods, with values that varied between runs; the cause was not identified. No time of GTLS was measured here, and nothing in this paper is a statement about GTLS on the hardware it was written for.

6. DISCUSSION

The searches trade accuracy against run time, and the measurements of Sect. 4.4 say where each trade is safe. Binned mode is the fastest of the searches whose recovery is within 1.4 points of exact mode at a matched false-alarm rate, in the three synthetic classes and on the 12 quiet TESS stars, one sector each (Sect. 4.6 gives its time). It departs from TLS in χ^2 where the phase coverage is uneven (Table 11), but not in recovery on these light curves, which cover one sector each. The box search and the hybrid find as many planets as exact mode in white noise, within 1 point, and on the 12 quiet TESS stars, where every interval holds zero, so on such stars the measurements do not separate the four searches and the choice is one of run time; they serve as a faster first pass, with exact TLS for confirmation. In correlated noise they find 5 to 6 points fewer and on a rotating star 31 points fewer, so binned or exact mode should be used there. The hybrid evaluates the TLS statistic only near the five highest peaks of the box spectrum, and refining 20 peaks in place of 5 changes its share by at most 0.3 points, pooled over the signal-to-noise ratios; its losses are the box search's, since the SDE of each peak is the box search's. In the variable class the cause is measured: that SDE is a z-score of a spectrum that carries the trend of the rotation, which TLS's detrending removes; in red noise it was not measured.

At 2×10^5 points exact mode takes about the time of the default backend of TLS, fused-pl, on 16 threads (Sect. 4.6), so on this machine a search that moves from that backend to exact mode gains the exact statistic at no cost in time. The gain of exact mode over TLS fused does not grow with the number of points (Sect. 4.6). The share of trial epochs that reach the exact correlation stays below 0.5% of those tested in every period range and size (Table 16; counted by template length, it reaches 2.8% for the longest templates, Sect. 4.8), and the fold and the screening take more of the time of exact mode than the correlations do (Table 17). The tables do not separate the gain of the screening from the GPU's parallelism, and no measurement here does.

Against the searches named in the introduction, transitgpu differs in two ways. cuvarbase, GTLS and CETRA each need an NVIDIA GPU, whereas transitgpu runs through PyTorch and Triton and was run here on an AMD GPU with ROCm; it has not been run on NVIDIA hardware. Of the three, only GTLS ports TLS, and the TLS search of transitgpu keeps the TLS interface and results objects. The 48 shuffled copies that the box search searches in less time than one search with TLS fused are searched with the box statistic, so that comparison is of the cost of a threshold, not of two searches with one statistic. Whether the shuffles in one launch make per-star thresholds practical for a survey depends on the calibration of Sect. 4.5, made on synthetic light curves of one sector: calibrated for white noise, conservative for the red noise of the generator, and on a variable star a flag of the rotation, which shuffles in blocks of one day cannot reproduce.

The comparison with GTLS (Sect. 5) is one of design and of published times: exact mode reproduces the χ^2 of TLS where GTLS departs from it by design, the published times of GTLS in Table 10 come from another machine, other light curves and another statistic, and none of the searches of this paper was compared with GTLS on injected planets.

7. CONCLUSIONS

The box search searches a light curve of 2×10^5 points and 48 shuffled copies of it in one call in 9.79 s, less than one search with TLS fused on 16 threads, with the box statistic in place of that of TLS (Sect. 4.6). Its bin sums do not depend on the order of the additions (Sect. 2.3). Binned mode approximates the statistic of TLS fused and takes 1.48 s (Sect. 4.6). It finds injected planets within 1.4 points of exact mode at a matched false-alarm rate in three synthetic classes and on 12 quiet TESS stars, one sector each (Sect. 4.4). Exact mode reproduces the χ^2 of TLS fused to float64 rounding (Sects. 4.1 and 4.2) and takes 9.91 s, about the time of TLS fused-pl on 16 threads; its gain over TLS fused does not grow with the number of points (Sect. 4.6). The hybrid search, at its default of five peaks, takes 1.06 s and finds as many planets as exact mode in white noise and on quiet TESS stars, and fewer in correlated noise and on a rotating star (Sect. 4.4). The threshold from 48 block-shuffled copies is calibrated in white noise, conservative in red noise, and marks the rotation of a variable star as signal (Sect. 4.5). GTLS, an earlier CUDA port of TLS, approximates the TLS statistic with float32 sums and a coarse first pass over the trial epochs, and exact mode reproduces it. The times that GTLS publishes come from another GPU and other light curves and are not compared with ours as a speed-up (Sect. 5).

These results have limits. The recovery is measured on synthetic light curves of one sector and on one sector of each of 12 real TESS stars chosen quiet, at one cadence; stars with rotation or other variability, and longer baselines, were not tested with real data, and the transits injected into the real stars have the durations of a solar-density star on every star. The hybrid search is a way to generate candidates, scored by any of its refined peaks, and not a search scored

by its best peak. The cause of the conservative threshold of the red class is not measured, and the loss that remains between the detrended box search and exact mode in the variable class is not explained. The package has been run only with ROCm on one AMD GPU; on other devices it uses only its Triton kernels, which Triton also compiles for NVIDIA GPUs, and its one HIP kernel, a faster accumulation of the box search, uses no AMD-specific instruction, but neither the speed nor the results on NVIDIA hardware have been measured. Every search here, as TLS and GTLS, folds at a strict ephemeris, so a planet whose transit times vary (transit timing variations) is folded out of alignment and its signal is lowered; none of the recovery tests injects such a planet. The package runs on one device: a call searches on one GPU, and the search of many light curves is one launch, not a division of the work across devices. All timings come from one machine and one strong, evenly sampled signal (3 000 ppm in 1 000 ppm of white noise), the screening statistics from one batch of periods per range, and a time ratio between the box search and TLS is not the speed-up of one algorithm, since they compute different statistics.

SOFTWARE AVAILABILITY

transitgpu 0.3.0 (tag v0.3.0) is available at https://github.com/mauri870/transitgpu under the MIT licence. The repository holds the tests, the benchmark script (`benchmarks/bench.py`), the examples, and the scripts that produce the numbers and tables of this paper (`paper/scripts`). The period- and duration-grid code that the package copies from TLS is under the MIT licence of TLS (Hippke and Heller).

Software: transitgpu 0.3.0, Python 3.12.13, PyTorch 2.12.0+rocm7.2 (Paszke et al. 2019), Triton 3.7.0 (Tillet et al. 2019), ROCm 7.2.53211, NumPy 2.5.4 (Harris et al. 2020), Numba 0.68.0 (Lam et al. 2015), transitleastsquares 2.0.1 (Hippke & Heller 2019), astropy 8.0.2 (Astropy Collaboration et al. 2013, 2018, 2022), batman 2.5.3 (Kreidberg 2015), wotan 1.10 (Hippke et al. 2019)

An AI assistant helped with parts of the code and the writing; the author reviewed all of it. This paper includes data collected by the TESS mission. Funding for the TESS mission is provided by the NASA Science Mission Directorate. The data were obtained from the Mikulski Archive for Space Telescopes (MAST) at the Space Telescope Science Institute (STScI), which is operated by the Association of Universities for Research in Astronomy, Inc., under NASA contract NAS 5-26555. The TESS light curves are at MAST, doi:10.17909/t9-nmc8-f686. This paper includes data collected by the K2 mission. Funding for the K2 mission is provided by the NASA Science Mission Directorate. STScI is operated by the Association of Universities for Research in Astronomy, Inc., under NASA contract NAS 5-26555. This research has made use of the NASA Exoplanet Archive, which is operated by the California Institute of Technology, under contract with the National Aeronautics and Space Administration under the Exoplanet Exploration Program.

APPENDIX

A. TABLES AND FIGURES OF THE VALIDATION AND THE TIMING

Table 11 gives the best period and SDE of binned mode and of TLS fused-pl against TLS fused on the 14 characterization cases (Sect. 2.6); Table 12 lists the fields of the results of TLS with the largest differences between exact mode and TLS fused on the 14 characterization cases (Sect. 4.1); Table 14 the share of the continuous box statistic that the box search keeps (Sect. 4.3); Table 15 what each timed row of Table 6 includes (Sect. 4.6); Table 16 the trial epochs that survive each bound of the screening cascade and Table 17 the GPU time of exact mode by stage (Sect. 4.8). Figure 6 pictures the calibration of Sect. 4.5 and Figure 7 the two tables of Sect. 4.8.

Table 11. Best period and SDE of TLS binned and TLS fused-pl against TLS fused on the 14 characterization cases.

case	binned period	binned |ΔSDE|	fused-pl period	fused-pl |ΔSDE|
`syn_earth`	same	0.022	same	0.001
`syn_earth_gap`	different	0.224	same	0.006
`syn_earth_dy`	unequal errors, refused		same	0.004
`no_fit`	same	0.000	same	0.000
`k2_box`	same	0.615	same	0.006
`k2_grazing`	same	0.658	same	0.047
`k2_3_window`	same	0.870	same	0.000
`k2_3_threads1`	same	0.245	same	0.000
`short_lc`	different	0.209	same	0.000
`centered_time`	same	0.139	same	0.008
`custom_nonlinear`	same	0.172	same	0.007
`custom_ecc_squareroot`	same	0.179	same	0.011
`narrow_stellar`	unequal errors, refused		same	0.065
`plato_25s_window`	same	0.003	same	0.002

Table 12. Largest difference between exact mode on the GPU and TLS fused over the 14 characterization cases, for the 10 fields with the largest differences.

field	relative	absolute	case
`power_raw`	1.0e-07	4.3e-10	`narrow_stellar`
`power`	7.9e-08	4.3e-10	`syn_earth_dy`
`rp_rs`	3.2e-10	2.7e-12	`syn_earth_dy`
`SR`	1.8e-11	1.4e-11	`syn_earth_dy`
`chi2`	1.8e-11	2.1e-07	`syn_earth_dy`
`chi2red`	1.8e-11	1.6e-11	`syn_earth_dy`
`SDE`	7.5e-12	4.2e-11	`syn_earth`
`SDE_raw`	7.5e-12	4.2e-11	`syn_earth`
`chi2_min`	2.2e-12	2.2e-08	`syn_earth`
`chi2red_min`	2.2e-12	1.7e-12	`syn_earth`

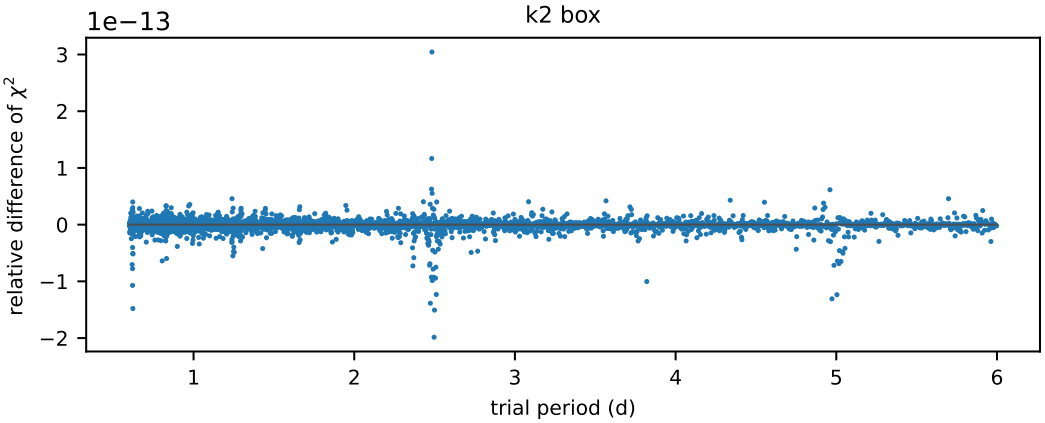


Figure 5. Relative difference of the χ^2 of exact mode on the GPU and of TLS fused at each trial period of the characterization case `k2_box`: the line is zero.

Table 13. Largest relative difference of the χ^2 of exact mode on the GPU and of TLS fused over the trial periods of each of the 14 characterization cases.

case	trial periods	largest difference
syn_earth	278	1.5×10^{-11}
syn_earth_gap	111	1.7×10^{-11}
syn_earth_dy	166	1.8×10^{-11}
no_fit	278	6.4×10^{-15}
k2_box	5222	3.0×10^{-13}
k2_grazing	5222	8.3×10^{-13}
k2_3_window	1709	9.2×10^{-13}
k2_3_threads1	296	7.4×10^{-13}
short_lc	1716	3.4×10^{-13}
centered_time	746	1.9×10^{-13}
custom_nonlinear	746	1.6×10^{-13}
custom_ecc_squareroot	746	2.3×10^{-13}
narrow_stellar	2590	7.5×10^{-13}
plato_25s_window	47	6.5×10^{-14}

Table 14. Share of the statistic of an exhaustive continuous box search kept by the box search, over 200 trial periods of Gaussian noise: median, 5th percentile and minimum.

bins	step	median	5th percentile	smallest
3	1.5	0.798	0.637	0.535
3	1.1	0.834	0.693	0.535
12	1.5	0.886	0.749	0.633
12	1.1	0.929	0.809	0.665

Table 15. What the timed call of each row includes. Box: the box search with the period grid given. astropy: `BoxLeastSquares.power`, with the object built before the timer. TLS: `power()` of TLS 2.0.1 on the CPU. transitgpu.tls: `power()` of `transitgpu.tls` on the GPU.

	box	astropy	TLS	transitgpu.tls
validation and cleaning of the input	yes	no	yes	yes
copy of times and flux to the GPU	yes	n.a.	n.a.	yes
period grid	no	no	yes	yes
duration grid and templates	yes	no	yes	yes
fold	yes	yes	yes	yes
search over periods	yes	yes	yes	yes
copy of results back, synchronization	yes	n.a.	n.a.	yes
SDE and period uncertainty	yes	no	yes	yes
fit at the best period	no	no	yes	yes

Table 16. Trial epochs remaining after each cascade bound, as a share of those tested, for one batch of 64 periods in each period range. Live: the box is deeper than the shallowest depth fitted. The trial epochs that pass U_3 get the exact float64 correlation; the last column gives their number per period.

points	period (d)	tested (10^6)	live (%)	U_1 (%)	U_2 (%)	U_3 (%)	per period
20 000	0.5 to 1.0	2.41	98.3	95.2	23.1	0.177	66
20 000	1.0 to 2.0	3.93	97.2	90.4	13.1	0.0914	56
20 000	2.0 to 5.0	7.71	95.4	84.2	11.4	0.0805	97
20 000	5.0 to 14.0	17.26	89.2	64.3	4.47	0.0441	119
100 000	0.5 to 1.0	2.28	100	100	41.9	0.311	111
100 000	1.0 to 2.0	3.63	100	99.8	23.5	0.152	86
100 000	2.0 to 5.0	7.04	99.9	97.1	14.5	0.126	138
100 000	5.0 to 14.0	14.38	99.1	88.6	10.5	0.0631	142
200 000	0.5 to 1.0	2.25	100	100	47.9	0.466	164
200 000	1.0 to 2.0	3.60	100	100	29.1	0.251	141
200 000	2.0 to 5.0	6.95	100	99.1	15.1	0.0915	99
200 000	5.0 to 14.0	14.04	99.9	91.1	9.61	0.0371	82

Table 17. GPU time in seconds, with its share of the total, of exact mode over the period grid of TLS by stage, from the profiler.

points	GPU total	fold	scans	screening	correlations	other
20 000	2.16	0.38 (18%)	0.26 (12%)	1.30 (60%)	0.06 (3%)	0.16 (7%)
100 000	4.78	1.31 (27%)	1.22 (26%)	1.73 (36%)	0.33 (7%)	0.20 (4%)
200 000	8.88	2.59 (29%)	2.47 (28%)	2.55 (29%)	0.88 (10%)	0.39 (4%)

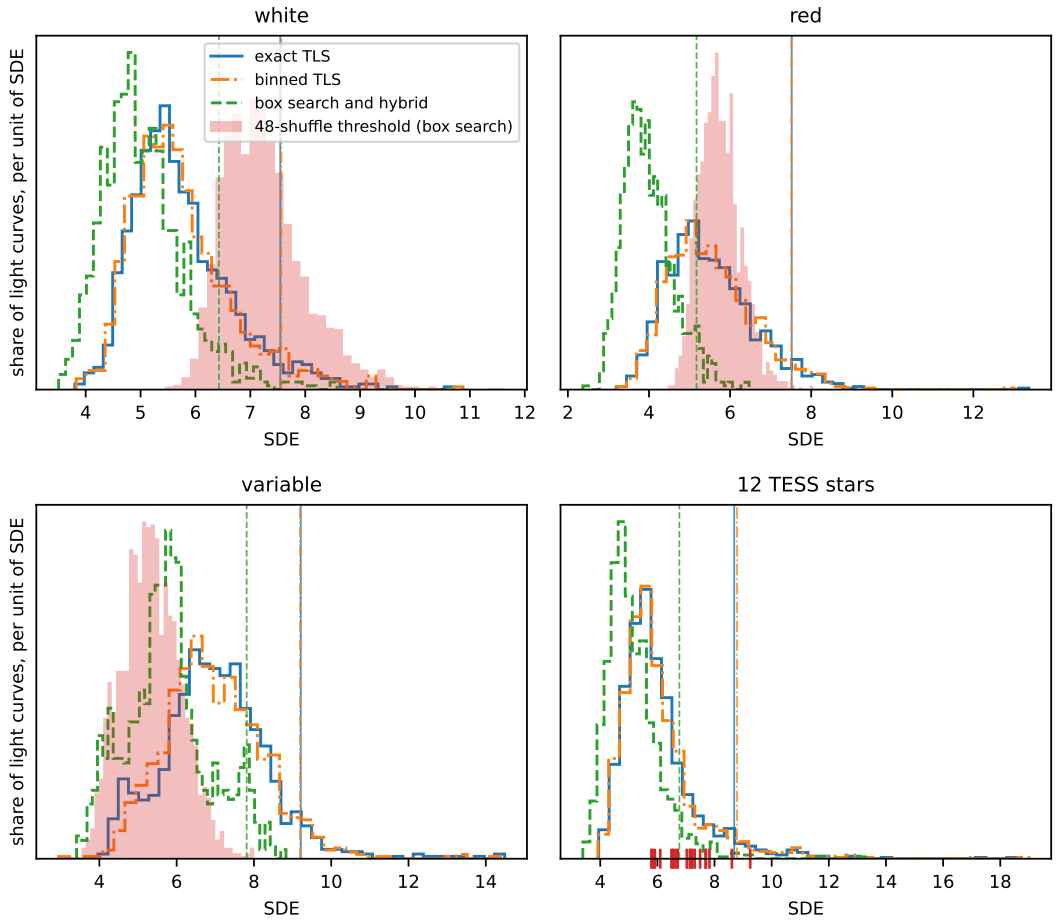


Figure 6. The SDE of the noise-only light curves of the recovery test (Sect. 4.4) for exact mode, binned mode and the box search, in each noise class and on the 12 real TESS stars, with the threshold of each search at a false-alarm rate of 5% as a vertical line of its colour; the hybrid has the SDE of the box search. The shaded distribution is the 48-shuffle threshold of the box search from the calibration light curves of Table 4, which are a different set from the same generator; on the real stars each mark is one star's threshold, the 12 stars of Sect. 3.3 and the eight further stars of the exchangeability test.

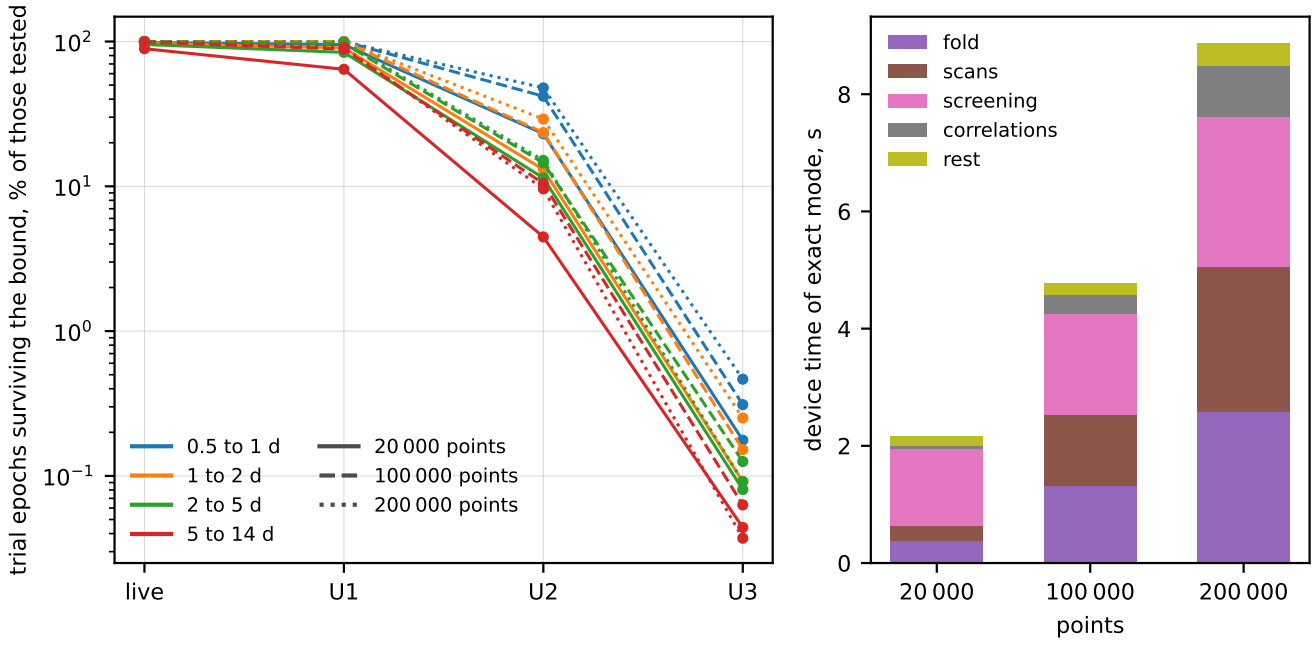


Figure 7. Left: the share of the trial epochs that survive each bound of the screening cascade (live, U_1, U_2, U_3) in each period range at the three sizes of Table 16, summed over the template lengths. Right: the GPU time of exact mode over the period grid by stage, from the profiler, at the three sizes of Table 17.

B. REPRODUCIBILITY

Every number measured for this paper comes from a command of the repository, run from its root with the GPU group installed (`uv sync --group rocm`; the figure scripts also need the `examples` extra). The commands that run a search need the GPU; the scripts that write the tables, the macros and the PDF from the recorded files do not.

- Agreement with TLS on its 14 characterization cases, and Table 12: `uv run pytest tests/test_tls_golden.py` and `uv run python paper/scripts/validation.py`.
- The adversarial cases of Sect. 4.2: `uv run pytest tests/test_tls_adversarial.py`, which runs TLS fused on the CPU as the reference, and `uv run python paper/scripts/adversarial.py`, which writes the largest differences measured to `paper/adversarial_table.json`.
- Table 11: `uv run python paper/scripts/binned_accuracy.py`.
- Table 16 and the share of trial epochs that reach the exact correlation: `taskset -c 0-15 uv run python paper/scripts/screening_stats.py --json paper/scripts/screening_stats.json`. Table 17: `taskset -c 0-15 uv run python paper/scripts/profile_split.py --json paper/scripts/profile_split.json`.
- Table 6, the times of Sect. 4.6 and the rows of Sect. 5.3: one session of `bash benchmarks/session.sh DIRECTORY`, which runs `benchmarks/bench.py` once for each part (the rows of the table, the set of 64 light curves, the shuffles, the light curves of the GTLS benchmark at both trial-epoch margins, the first call of a fresh process of Sect. 4.6 and the profile of Table 17, by the same script as the item above), then `uv run python benchmarks/assemble.py DIRECTORY --out benchmarks/results.json`. The rows that time only the CPU, and the exact-mode row of TLS at the input of the GTLS paper, were not run again for the version of this paper; their files are those of the earlier session, put with the parts of the new one before `assemble.py` runs, each such row marked with its session and its own versions; the command, the rows and the number of calls of every part are recorded in that file, and the synthetic light curves are made by the script with the seed 20260101. Table 17 itself: `uv run python paper/scripts/split_table.py paper/scripts/profile_split.json`.
- Fig. 2 and Table 2: `uv run python validation/run.py PART DIRECTORY` for each part (main-white, main-red and main-variable) and `uv run python validation/analyse.py DIRECTORY`; binned mode on the same light curves by `uv run python validation/rerun_binned.py PART RERUN_DIRECTORY` for each part and `uv run python validation/analyse_binned.py RERUN_DIRECTORY DIRECTORY`, put in place of the first binned rows by `uv run python paper/scripts/merge_recovery.py DIRECTORY/results.json RERUN_DIRECTORY/results-bands.json --out paper/recovery.json`; then the real part by `uv run python validation/run.py real DIRECTORY_REAL` and `uv run python validation/analyse.py DIRECTORY_REAL`, whose `results.json` is `paper/recovery_real.json`; then `uv run python paper/scripts/completeness.py paper/recovery.json --real paper/recovery_real.json`.
- Tables 4 and 5: `uv run python validation/threshold.py` for the rows (the modes null, inj and real; the red and variable classes are flattened first by its `prepare` mode into a directory given as `--cache`, and the real mode fetches the light curves of `validation/stars.json` through `validation/fetch.py`) and `uv run python validation/threshold_analysis.py ROWS --json paper/threshold.json`; then `uv run python paper/scripts/threshold_table.py paper/threshold.json` and `uv run python paper/scripts/fill_numbers.py`.
- Table 3: `uv run python paper/scripts/period_table.py paper/recovery.json --real paper/recovery_real.json` (no GPU).
- Fig. 3: `uv run python validation/roc.py DIRECTORY --bands RERUN_DIRECTORY --real DIRECTORY_REAL` on the directories of Fig. 2, which writes `paper/roc.json`, then `uv run python paper/scripts/roc.py` (no GPU).
- Table 14: `uv run python paper/scripts/bins_share.py`.
- The three box searches on the variable class (Sect. 4.4): `uv run python validation/box_variable.py OUTPUT_DIRECTORY DIRECTORY --out paper/box_variable.json`, on the evaluation directory of Fig. 2.

- The attained column of Table 9: `uv run python benchmarks/peak.py --spec-bandwidth 960 --json paper/peak.json`, with the device otherwise idle.
- Fig. 4: `uv run python examples/plot_wasp18.py`, on `examples/data/wasp18_s2.npz`.
- The macros of the paper: `uv run python paper/scripts/fill_numbers.py` writes `paper/numbers.tex` from `benchmarks/results.json`, `paper/split_numbers.tex` from `paper/scripts/profile_split.json`, `paper/recovery_numbers.tex` from `paper/recovery.json` and `paper/threshold_numbers.tex` from `paper/threshold.json` and `paper/peak_numbers.tex` from `paper/peak.json`, and `uv run pytest tests/test_paper_numbers.py` checks that they agree with their sources (no GPU).
- The PDF: `uv run python paper/build.py` (no GPU).

The software versions and the hardware are recorded in `results.json` and in Sect. 3.5.

REFERENCES

Advanced Micro Devices. 2026, AMD Radeon RX 7900 XTX, Product support page, https://www.amd.com/en/support/graphics/amd-radeon-rx-7000-series/amd-radeon-rx-7900-series/amd-radeon-rx-7900xtx, accessed 2026-10-25

Akeson, R. L., Chen, X., Ciardi, D., et al. 2013, PASP, 125, 989, doi: 10.1086/672273

Astropy Collaboration, Price-Whelan, A. M., Lim, P. L., et al. 2022, ApJ, 935, 167, doi: 10.3847/1538-4357/ac7c74

Astropy Collaboration, Price-Whelan, A. M., Sipőcz, B. M., et al. 2018, AJ, 156, 123, doi: 10.3847/1538-3881/aabc4f

Astropy Collaboration, Robitaille, T. P., Tollerud, E. J., et al. 2013, A&A, 558, A33, doi: 10.1051/0004-6361/201322068

GPU-Monkey. 2026, AMD Radeon RX 7900 XTX vs AMD Radeon Pro W7900, Specification database, https://www.gpu-monkey.com/en/compare_gpu-amd_radeon_rx_7900_xtx-vs-amd_radeon_pro_w7900, accessed 2026-10-26

Harris, C. R., Millman, K. J., van der Walt, S. J., et al. 2020, Nature, 585, 357, doi: 10.1038/s41586-020-2649-2

Hartman, J. D., & Bakos, G. Á. 2016, Astronomy and Computing, 17, 1, doi: 10.1016/j.ascom.2016.05.006

Hellier, C., Anderson, D. R., Collier Cameron, A., et al. 2009, Nature, 460, 1098, doi: 10.1038/nature08245

Hippke, M., David, T. J., Mulders, G. D., & Heller, R. 2019, AJ, 158, 143, doi: 10.3847/1538-3881/ab3984

Hippke, M., & Heller, R. 2019, A&A, 623, A39. https://arxiv.org/abs/1901.02015

Hoffman, J. 2022, cuvarbase: Fast period finding utilities for GPUs, Astrophysics Source Code Library, record ascl:2210.030. https://ascl.net/2210.030

Howell, S. B., Sobeck, C., Haas, M., et al. 2014, PASP, 126, 398, doi: 10.1086/676406

Hu, Q., Ge, J., Jin, L., & Willis, K. 2026, GTLS: A GPU-accelerated method for periodic transit detection. https://arxiv.org/abs/2607.00348

Jenkins, J. M., Twicken, J. D., McCauliff, S., et al. 2016, in Proc. SPIE, Vol. 9913, Software and Cyberinfrastructure for Astronomy IV, 99133E, doi: 10.1117/12.2233418

Kovács, G., Zucker, S., & Mazeh, T. 2002, A&A, 391, 369. https://arxiv.org/abs/astro-ph/0206099

Kreidberg, L. 2015, PASP, 127, 1161, doi: 10.1086/683602

Kunimoto, M., Bryson, S., Daylan, T., et al. 2023a, Research Notes of the AAS, 7, 7, doi: 10.3847/2515-5172/acb149

Kunimoto, M., Tey, E., Fong, W., et al. 2023b, QLP Data Release Notes 003: GPU-based Transit Search. https://arxiv.org/abs/2302.01293

Kunimoto, M., Bryson, S., Jaffee, D., et al. 2025. https://arxiv.org/abs/2509.10619

Lam, S. K., Pitrou, A., & Seibert, S. 2015, in Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC, 7:1–7:6, doi: 10.1145/2833157.2833162

Notebookcheck. 2026, AMD Radeon RX 7900 XTX GPU benchmarks and specs, https://www.notebookcheck.net/AMD-Radeon-RX-7900-XTX-GPU-Benchmarks-and-Specs.674159.0.html, accessed 2026-10-25

NVIDIA Corporation. 2022, NVIDIA Ada GPU Architecture, Whitepaper, https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf, accessed 2026-10-25

—. 2026, GeForce RTX 4090, Product page, https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/rtx-4090/, accessed 2026-10-25

Ofir, A. 2014, A&A, 561, A138. https://arxiv.org/abs/1307.7330

Paszke, A., Gross, S., Massa, F., et al. 2019, in Advances in Neural Information Processing Systems 32, 8024–8035. https://arxiv.org/abs/1912.01703

Pont, F., Zucker, S., & Queloz, D. 2006, MNRAS, 373, 231, doi: 10.1111/j.1365-2966.2006.11012.x

Ricker, G. R., Winn, J. N., Vanderspek, R., et al. 2015, Journal of Astronomical Telescopes, Instruments, and Systems, 1, 014003, doi: 10.1117/1.JATIS.1.1.014003

Smith, L. C., Ahmed, S., De Angeli, F., et al. 2025, MNRAS, 539, 297. https://arxiv.org/abs/2503.20875

Tango, T. 1998, Statistics in Medicine, 17, 891, doi: 10.1002/(SICI)1097-0258(19980430)17:8<891::AID-SIM780>3.0.CO;2-B

Thompson, S. E., Coughlin, J. L., Hoffman, K., et al. 2018, ApJS, 235, 38. https://arxiv.org/abs/1710.06758

Tillet, P., Kung, H. T., & Cox, D. 2019, in Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL 2019), 10–19, doi: 10.1145/3315508.3329973